ZUVY · BACKEND, FRONTEND, QA & DEVOPS GUIDE

Student ID Login: Guide for Backend, Frontend, QA and DevOps

Branch: feature/student-id-login · Date: 27 October 2026 · Audience: backend and frontend developers, testers, DevOps, programme team

Many students (classes 5–10) don't have an email address, so they can't use "Sign in with Google". This feature lets them **create an account with their name, a password and their school** (picked from dropdowns: state → city → school). The system then gives them a **Student ID** made from their name (for example `riyasharmact1`), which they use with their password to log in.

After logging in, these students can do **everything a Google-login student can do**: open courses, watch videos, take quizzes and coding problems, see progress and the leaderboard. Nothing changes for users who log in with Google, except that **every login now gets a session ID** (Part F9).

Item	Value
Backend code	Branch `feature/student-id-login`, to be merged into `chore/development_server_new`
Dev API	`https://dev.api.zuvy.org`
Dev Swagger	`https://dev.api.zuvy.org/apis`
Dev frontend	`https://dev.app.zuvy.org`
New APIs	5 under `/auth/student`, 4 under `/schools` (3 public dropdown lists + 1 super admin CSV upload)
Migrations	`0043`, `0044`, `0045` (run in this order, Part C1)
School list	13,72,195 schools from the official UDISE file, loaded once with one command (Part C8)

Who should read what

You are	Read these parts
Everyone	**Part A** (the big picture, in simple words)
Frontend developer	Part A, **Part D** (what to build, the school dropdowns and the Cloudflare token), Part F1–F2 and F7 (API details), F9 (session ID)
Backend developer	Part A, Part B (what changed), **Part E** (testing in Swagger), Part F (API reference)
Tester (QA)	Part A, **Part G** (test plan), Part H (database checks), Part J (troubleshooting)
DevOps / server owner	Part A, **Part C** (server setup, migrations, loading the school list)
Super admin	Part A6 (where the school list comes from) and Part C9 (adding a school a student can't find)

Contents

- Part A: The big picture in simple words
- Part B: What was built in the backend

- Part C: Server setup for DevOps
- Part D: Frontend guide
- Part E: Backend developer guide: testing in Swagger
- Part F: API reference
- Part G: Tester guide
- Part H: Checking the database
- Part I: Known limitations
- Part J: Troubleshooting

Part A: The big picture in simple words

A1. The problem

- Today every user signs in with Google, which needs an email address.
- Many young students don't have an email, so they couldn't use Zuvy at all.

A2. The solution

- A student opens the sign-up page, types **their name and a password**, and picks **their state, then their city, then their school** from three dropdowns.
- The system creates their account, saves the school's name and **UDISE code** with it, and shows them a **Student ID** made from their name, such as `riyasharmact1` for "Riya Sharma".
- From then on they log in with **Student ID + password**.
- Behind the scenes they are normal students, so **all courses and features work for them exactly like for Google users**.
- **There is no "forgot password" button.** If a student forgets their password, they ask the team. A **super admin** (a developer) sets a new password with an admin API and gives it to the student.

A3. What Cloudflare does (and why we need it)

Because sign-up needs no email and no phone number, a bad actor could write a small program that creates **thousands of fake accounts** in minutes. We need a way to tell **real people** apart from **programs (bots)**.

Cloudflare Turnstile is a free "Are you human?" check, a modern replacement for the old picture puzzles. Think of it as a **security guard at the door**:

1. **The guard checks the visitor.** When the sign-up page opens, a small Cloudflare box appears. Cloudflare checks the browser quietly in the background. Most students see **nothing, or just one checkbox**. No picture puzzles, which suits young children.
2. **The guard gives a one-time pass.** If the visitor looks human, Cloudflare gives the page a **token**: a long random text that works as a one-time pass.
3. **The pass is shown at the counter.** When the student clicks "Create account", the frontend sends the token to our backend together with the name and password.
4. **The counter calls the guard to confirm.** Our backend asks Cloudflare, "Is this pass real?" using a **secret key** that only the server knows. Only if Cloudflare says "yes" is the account created.

Important facts about the token:

- It works **only once** and **expires after 5 minutes**.

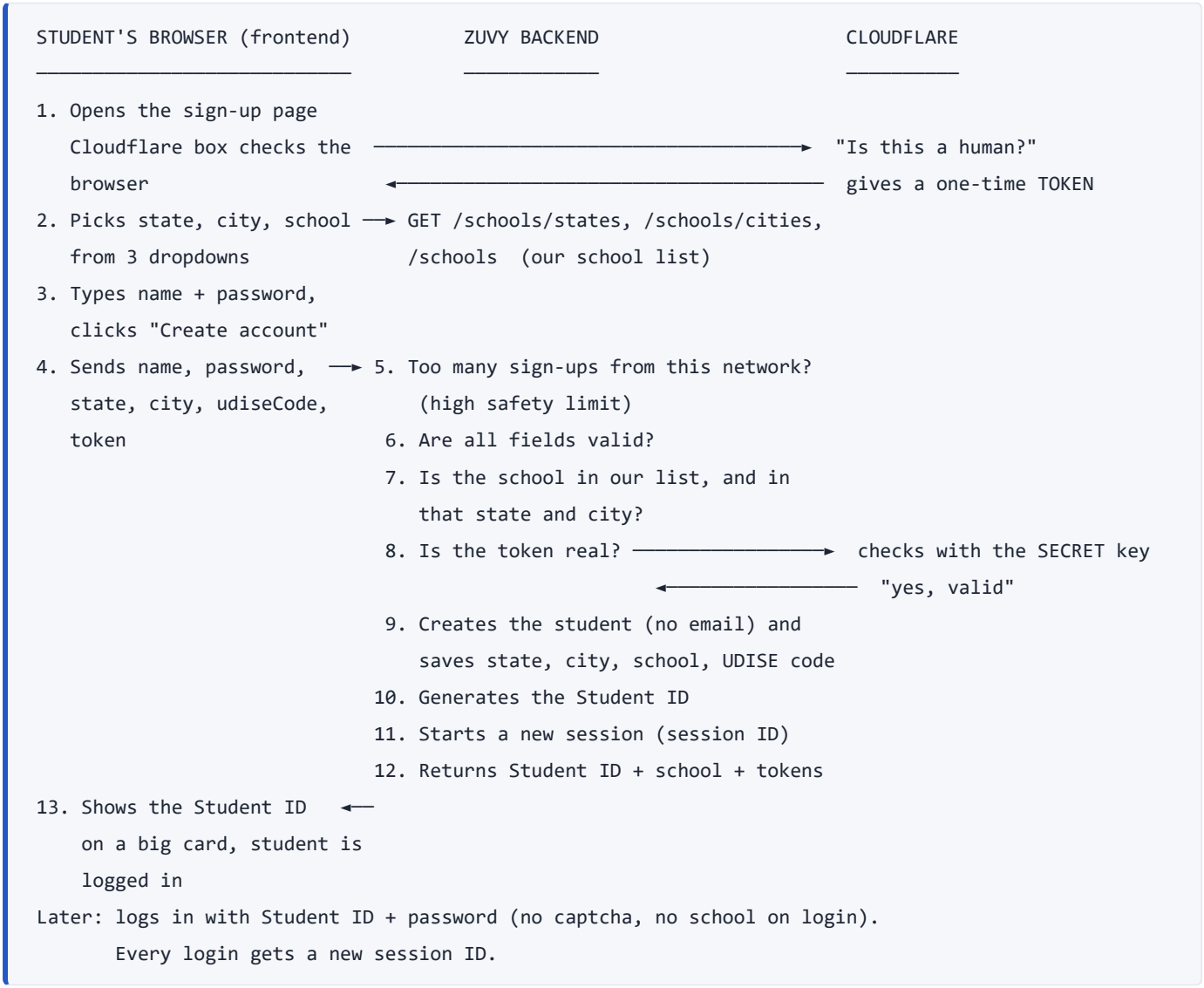
- It only proves "a human is here". It is **not a login token**, and it is never stored.
- A bot that calls our API directly has no valid token, so it is refused.

Why not just block by IP address? In a school, 40 students share **one Wi-Fi**, so they all have the **same public IP address**. A strict per-IP limit would block real students. Cloudflare judges **each browser separately**, so every student in the lab passes on their own. We still keep a **high per-IP limit**, but only as a safety ceiling against floods (Part B1).

There are **two keys** from Cloudflare:

Key	Who uses it	Secret?	Where it goes
Site key	Frontend	No, it's public	Frontend config, used to show the box
Secret key	Backend only	**Yes**, never share or commit	Server `.env` as `TURNSTILE_SECRET_KEY`

A4. What happens, step by step



A5. Status: what is done and what is left

Area	Work	Status	Owner
Backend	Sign-up, login, Student ID, password hashing, lockout	**Done**	Backend

Area	Work	Status	Owner
Backend	Super admin tools: reset password, find student	**Done**	Backend
Backend	Enrol a student by Student ID into a course and batch: super admins (any course), **org admins and ops** (their own organisation's courses), recorded in the activity log	**Done**	Backend
Backend	Student IDs made from the name (`riyasharmact1` , `riyasharmact2` , `riyact1`), Hindi names spelled in English letters, atomic per-name counter, migration `0044`	**Done**	Backend
Backend	State, city and school (with UDISE code) at sign-up; school dropdown APIs with word search; migration `0045`	**Done** (not yet committed)	Backend
Backend	One-command import of the official UDISE school list (13,72,195 schools, checked against the real file), and a CSV upload to add a missing school	**Done** (not yet committed)	Backend
Backend	A new **session ID** for every login (Google and Student ID), kept on token refresh and org switch, closed on logout; migration `0045`	**Done** (not yet committed)	Backend
Backend	Cloudflare token check, per-IP safety limits, real client IP behind nginx	**Done**	Backend
Backend	Fixes so email-less students don't break classes, attendance or login checks	**Done**	Backend
Backend	Unit tests (87 tests), and an end-to-end check on a real Postgres with 1.49 lakh schools from the real file	**Done**	Backend
Server (dev)	nginx forwards the real client IP	**Done**	DevOps
Server (dev)	New `.env` values added	**Done**	DevOps
Server	Run database migrations `0043` , `0044` and **`0045`** **(before deploying this code)**, bind the container port to localhost, check outbound access to Cloudflare (Part C)	To do / confirm	DevOps
Server	**Load the school list once** with one command (Part C8). Without it the dropdowns are empty and nobody can sign up with a Student ID.	**To do**	Backend / DevOps
Cloudflare	Create the Turnstile widget and get the site key and secret key (Part C2)	To do / confirm	DevOps / lead
Frontend	Sign-up page with the state / city / school dropdowns and the Cloudflare box, Student ID login, Student ID card, error messages (Part D)	**To do**	Frontend
Frontend	Admin screen: "Add student by Student ID" in a course's students page (Part D12)	**To do**	Frontend
Product	Decide which **extra student details** to collect after sign-up (not required at sign-up). The list was meant to come as a separate PDF, which hasn't been received yet.	**Waiting**	Product
QA	Test plan (Part G)	**To do** after deploy	QA

A6. The school list and the UDISE code

Every school in India has a **UDISE code**, an 11-character government ID for the school (for example `36070700101`). Saving the UDISE code, and not only the school's name, lets us count students per school reliably, even when two schools have similar names.

There is no public API for this. We checked:

- `github.com/gsidhu/udise-csv-data` and `github.com/gsidhu/udise-district-dump` only have **summary statistics** per state and district: no school names, no UDISE codes, no API.
- The official UDISE+ portal shows one school at a time behind a captcha. It isn't an API, and **we don't call it from our server**. (Paid "UDISE API" services online are unofficial scrapers of it.)

Where the list comes from. The **India Data Portal** publishes the Ministry of Education's UDISE data as a downloadable file: dataset **"UDISE"**, file **"Basic Details of Schools"** (CSV, about 400 MB). It lists **13,72,195 operational schools** in 35 states and union territories, with UDISE code, school name, state, district, block, village, pincode, management (government, private and so on) and the classes taught.

A developer loads this file **once per environment** with one command (Part C8). Our server then answers the dropdowns from its own table `zuvy_schools`, so sign-up is fast and doesn't depend on any government website. When a newer file comes out (about once a year), run the same command again: schools are matched by UDISE code and updated, never duplicated.

What the super admin does: almost nothing. Only if a student's school is missing (for example a school opened after the file was made), the super admin adds it with a one-line CSV upload (Part C9).

City means district. UDISE groups schools by **district**, so the "city" dropdown shows districts (for example `Hyderabad`, `Patna`). States and districts keep the file's spelling, such as "Andaman and Nicobar Islands"; the APIs accept them in any letter case.

Schools only, for now. Colleges have no UDISE code; they can be added later with a separate list.

A7. Session ID

Every time **any** user logs in (Google or Student ID), the server creates a new **session ID**, a random ID like `6f1c2b8e-4d7a-4b9e-9c1a-2f3d4e5a6b7c`.

- It is returned in the login response as `sessionId` and is also inside the JWT token as `sid`.
- It **stays the same** when the token is refreshed or the user switches organisation, because that is still the same login.
- It is **closed** when the user logs out.
- Each session is saved in the table `zuvy_user_sessions`: who, when, how they logged in (`google`, `student_id`), the IP address and the browser.

This lets us count logins and see how long people stay. Saving the session is **best effort**: if it ever fails, the login still works and the error is only logged.

Part B: What was built in the backend

B1. The rules in simple words

- **Same user table:** the student is a normal row in `users`, with `email` and `google_user_id` both empty (NULL) and `mode = 'student'`.

- **Separate login table:** the Student ID and the password hash live in a new table, `zuvy_student_credentials`, linked to `users.id`.
- **Student ID format:** first name + last name in lowercase, then `ct` and a number, with no leading zero. "Riya Sharma" → `riyasharmact1`; the next "Riya Sharma" → `riyasharmact2`; "Riya" with no last name → `riyact1`. The number is counted **per name**, so IDs stay short even with thousands of students (a very common name might reach `ravict900`). Middle names are skipped ("Riya Kumari Sharma" → `riyasharmact1`), accents become plain letters, and spaces, dots, hyphens, apostrophes and digits are dropped. **Hindi (Devanagari) names** are spelled in English letters for the ID only: रवि → `ravict1`, अरुणेश धर → `aruneshdharct1`. The name itself is still saved and shown exactly as typed (रवि). Names in other scripts get `studentct1`. IDs are stored in lowercase and login accepts any case (`RiyaSharmaCT1` works too).
- **Numbers never clash:** each name has a counter in the `zuvy_student_id_counters` table, increased by one atomic database statement. Two students with the same name signing up at the same moment always get different numbers, and a number is never reused, even if an account is deleted. A unique rule on `student_id` stays as a safety net.
- **Passwords:** 6–64 characters. Never stored as plain text, only as a salted `scrypt` hash.
- **Lockout:** after **5 wrong passwords**, that one account is locked for **15 minutes**. Classmates on the same Wi-Fi are not affected.
- **Captcha:** sign-up needs a valid Cloudflare token (Part A3). Login has no captcha.
- **Per-IP safety limits**, only on the two Student ID routes (all other APIs have no limit):

Route	Default limit per IP	Can be changed in `.env` with
Sign-up	60 per minute and 300 per hour	`SIGNUP_LIMIT_PER_MINUTE`, `SIGNUP_LIMIT_PER_HOUR`
Login	600 per 15 minutes	`LOGIN_LIMIT_PER_15_MINUTES`

  These are high on purpose: a whole school lab can sign up at the same time without hitting them.
- **Courses:** Public courses enrol the student automatically when they open one, exactly as for Google students. For **private** courses, a **super admin**, or an **admin / ops** of the course's organisation, enrols them into a batch by Student ID (Part F5). The usual "add students" API can't be used, because it works by email. Moving a student between batches (`PATCH /batch/reassign/...`) and the "unassigned students" list work unchanged, because they use the user id.
- **Same session as Google:** login returns the same `access_token` and `refresh_token`, made by the same code as Google login. `/auth/refresh` and `/auth/logout` work unchanged.
- **School at sign-up (required):** sign-up needs `state`, `city` and `udiseCode`, all from the school dropdowns (Part F7). The backend checks that the UDISE code is in our school list, that the school is active, and that it really is in the chosen state and city (any letter case is accepted). It then saves the **school list's own spelling** of the state, city (district) and school name, plus the UDISE code, on the student's `zuvy_student_credentials` row. It's a copy, so the student's record stays the same even if the list changes later. The school is checked **before** the captcha, so a wrong pick doesn't use up the student's captcha token.
- **School list:** table `zuvy_schools`, loaded once from the official UDISE file with `scripts/import-schools.ts` (Part C8). A super admin can add a missing school with a CSV upload (Part C9, F8). Schools are matched by UDISE code, so loading again updates instead of duplicating. The three dropdown APIs are public, because sign-up happens before login.
- **UDISE codes** are 11 characters. Codes of states 01–09 often lose their leading zero in spreadsheets (`9010100101`); the import and the APIs add it back. A few real New Delhi schools have letters in their official

code (`070501ND201`), so codes may contain capital letters; any case is accepted.

- **Fast dropdowns:** the states and cities are kept in the server's memory for 10 minutes, so the state and city lists don't scan 13.7 lakh rows each time. Schools are read per city through an index sorted by name.
- **Session ID for every login (all users):** see Part A7 and F9. A new ID on every Google or Student ID login, the same ID after refresh and org switch, closed on logout.

B2. New files

File	What it is
`drizzle/migrations/0043_create_zuvy_student_credentials.sql`	Creates the `zuvy_student_credentials` table
`drizzle/migrations/0044_student_id_from_name.sql`	Name-based Student IDs: widens `student_id` to 40 characters, lowercases existing IDs, creates and back-fills the per-name counter table `zuvy_student_id_counters`
`src/student-auth/student-auth.module.ts`	New Nest module
`src/student-auth/student-auth.controller.ts`	The 5 new endpoints under `/auth/student`
`src/student-auth/student-auth.service.ts`	Sign-up, login, lockout, password reset, search and enrol logic
`src/student-auth/devanagari.ts`	Spells Hindi (Devanagari) names in English letters for the Student ID (रवि → ravi)
`src/student-auth/devanagari.spec.ts`	Unit tests for the Hindi spelling
`src/student-auth/dto/student-auth.dto.ts`	Request body and query validation
`src/student-auth/captcha.service.ts`	Checks the Cloudflare token with Cloudflare
`src/student-auth/student-auth.throttle.ts`	Per-IP limits for sign-up and login (read from `.env`)
`src/config/trust-proxy.ts`	Which proxies (nginx via Docker) are trusted for the real client IP
`src/student-auth/student-auth.service.spec.ts`	Unit tests: password hashing and Student ID generation
`src/student-auth/captcha.service.spec.ts`	Unit tests: the Cloudflare token check
`src/student-auth/student-auth.throttle.spec.ts`	Tests: per-IP limits behind a proxy, including faked headers
`drizzle/migrations/0045_schools_and_sessions.sql`	Creates `zuvy_schools` and `zuvy_user_sessions`; adds `state`, `city`, `school_name`, `udise_code` to `zuvy_student_credentials`. Safe to run twice.
`src/schools/schools.module.ts`	New Nest module for the school list

File	What it is
`src/schools/schools.controller.ts`	`GET /schools/states`, `GET /schools/cities`, `GET /schools` (public) and `POST /schools/admin/upload` (super admin)
`src/schools/schools.service.ts`	Dropdown queries (states and cities cached for 10 minutes, word search), school lookup by UDISE code, CSV upload
`src/schools/dto/schools.dto.ts`	Query validation for the dropdown APIs
`src/schools/csv.ts`	CSV reader that can read a 400 MB file piece by piece (quotes, commas and line breaks inside quotes, Windows and Unix line ends, Excel's byte order mark)
`src/schools/school-rows.ts`	Turns a CSV row into a school: reads the UDISE file's own column names, adds back dropped leading zeros, skips non-operational schools, keeps one spelling per city
`src/schools/school-store.ts`	Saves schools in batches of 1,000 (add new, update existing), shared by the import script and the upload API
`src/schools/udise-file.ts`	Streams the big UDISE file and hands good schools over in batches; counts and explains skipped rows
`scripts/import-schools.ts`	The one-time import command (Part C8). Only checks the file unless `--yes` is given.
`src/schools/school-rows.spec.ts`	Unit tests: CSV reading (including pieces split mid-quote and mid-line-end), row rules with the real file's columns, spellings
`src/schools/udise-file.spec.ts`	Unit tests: the importer on a small file in the real format (batches, state filter, duplicates, skipped rows)
`src/schools/schools.service.spec.ts`	Unit tests: upload is super admin only, batches, counts, spellings; dropdowns in any letter case; the 10-minute cache

B3. New table: `zuvy_student_credentials`

Column	Type	Meaning
`id`	serial	Primary key
`user_id`	bigint, unique, FK → `users.id` (cascade delete)	The student
`student_id`	varchar(40), unique, lowercase	Login ID, e.g. `riyasharmact1`
`password_hash`	varchar(255)	`scrypt$16384$8$1$<salt>$<hash>`
`failed_attempts`	integer, default 0	Wrong passwords in a row
`locked_until`	timestamptz, nullable	Login blocked until this time
`password_updated_at`	timestamptz	Last password change
`last_reset_by`	bigint, FK → `users.id`	Super admin who last reset the password
`last_reset_at`	timestamptz	When it was last reset
`created_at`	timestamptz	When the account was created
`state`	varchar(100), nullable (added in 0045)	State chosen at sign-up, e.g. `Telangana`
`city`	varchar(100), nullable (added in 0045)	City = UDISE district, e.g. `Hyderabad`

Column	Type	Meaning
`school_name`	varchar(255), nullable (added in 0045)	School name from the school list
`udise_code`	varchar(11), nullable, indexed (added in 0045)	The school's 11-character UDISE code

The four school columns are empty for accounts created before migration 0045. Every new sign-up fills them.

School list: `zuvy_schools` (one row per school, loaded from the UDISE file)

Column	Type	Meaning	From the UDISE file column
`id`	serial	Primary key	
`udise_code`	varchar(11), unique	UDISE code (leading zero added back)	`udise_school_code`
`school_name`	varchar(255)	School name (UDISE writes them in capitals)	`school_name`
`state`	varchar(100)	State, e.g. `Andaman and Nicobar Islands`	`state_name`
`district`	varchar(100)	District, shown as "city" in the app	`district_name`
`block`	varchar(100), nullable	Education block (blank for about a quarter of schools)	`subdistrict_name`
`village`	varchar(100), nullable	Village or town	`village_name`
`pincode`	varchar(6), nullable	Pincode (blank when unknown; the file's `0` is stored as blank)	`pincode`
`management`	varchar(100), nullable	e.g. `Department of Education`, `Private Unaided (Recognized)`, `Local body`	`management`
`class_from`	smallint, nullable	Lowest class taught (0 = pre-primary)	`class_from`
`class_to`	smallint, nullable	Highest class taught	`class_to`
`is_active`	boolean, default true	Inactive schools are hidden from the dropdowns and refused at sign-up	`status` = Operational
`created_at`	timestamptz	When the school was first loaded	
`updated_at`	timestamptz	When it was last changed by an import or upload	

Index on `(state, district, school_name)`: a city's schools come out already sorted by name. The full list takes about 430 MB in the database, including indexes (measured: about 325 bytes per school).

Sessions: `zuvy_user_sessions` (one row per login, all users)

Column	Type	Meaning
`id`	uuid, primary key	The session ID (`sessionId` in responses, `sid` in the JWT)
`user_id`	bigint, FK → `users.id` (cascade delete), indexed	The user
`login_method`	varchar(20)	`google`, `student_id`, or `token_refresh` / `org_switch` for tokens made before this change

Column	Type	Meaning
`org_id`	integer, nullable	Organisation at login
`ip`	varchar(64), nullable	Client IP (the real one, via nginx)
`user_agent`	varchar(512), nullable	Browser
`created_at`	timestamptz	Login time
`last_refreshed_at`	timestamptz, nullable	Last token refresh in this session
`ended_at`	timestamptz, nullable	Logout time; empty while the session is open or if the user never logged out

Counter table: `zuvy_student_id_counters` (one row per name)

Column	Type	Meaning
`base`	varchar(40), primary key	The name part, e.g. `riyasharma`
`last_number`	integer	Last number given out, e.g. `2` when `riyasharmact1` and `riyasharmact2` exist
`updated_at`	timestamptz	When the last number was given out

B4. Changes to existing files

Every change below is a **no-op for users who have an email**: they run exactly the same logic as before. Each change only matters when a user's email is empty. **The one exception is the session ID**, which is for all users: every login response gains a `sessionId` field and every login adds a row to `zuvy_user_sessions`. Nothing existing was removed or renamed, and a failure to save the session never blocks a login.

File	Change	Why it was needed
`src/auth/auth.service.ts`	The token-issuing half of Google `login()` moved into a new method, `issueLoginSession(user)`. Google login calls it unchanged.	Student ID login reuses exactly the same code, so both produce identical tokens.
`src/auth/auth.service.ts`	`isPoc` now also requires the user to have an email (login, refresh, org switch).	For email-less users, `null === null` would wrongly mark them as an organisation's point of contact.
`src/middleware/jwt.middleware.ts`	(a) `/auth/student/signup` and `/auth/student/login` added to the public routes. (b) A token with no email now matches the user on `email IS NULL`.	(a) Students can sign up and log in without a token. (b) The old check `email = <token email>` can never match an empty email in SQL, so every request from these students would have returned 401. Tokens with an email run the exact same SQL as before.
`src/controller/classes/classes.service.ts`	Students without an email are left out of Google Calendar and Zoom invite lists (class creation and session merge).	One empty attendee makes Google reject the whole event, which would break class creation for the entire batch.

File	Change	Why it was needed
`src/services/attendance-worker/attendance-worker.service.ts`	Zoom attendance matching ignores empty emails.	Zoom reports guests with an empty email; without this, a guest could be credited to a student who has no email.
`src/services/attendance/attendance-calculation.service.ts`	Google Meet attendance matching ignores empty emails.	Same reason as above.
`src/controller/student/student.service.ts`	`email.toLowerCase()` → `email?.toLowerCase() ?? null` in "completed classes with attendance".	It would have crashed for a student without an email.
`src/main.ts`	`app.set('trust proxy', ...)`, trusting only nginx's addresses (loopback and Docker's private ranges).	Without it, every request looks like it comes from nginx, so a per-IP limit would be one limit for everyone. No existing code reads the client IP, so other routes behave exactly as before.
`drizzle/schema.ts`	Added the `zuvyStudentCredentials` table. Also a type-only fix: the `bytea` custom type now declares `driverData: Buffer`.	Drizzle definition of the new table. The type fix removes an editor error; it changes nothing at runtime.
`drizzle/schema.ts`	Added `zuvySchools`, `zuvyUserSessions`, and the four school columns on `zuvyStudentCredentials`.	Drizzle definitions for migration 0045.
`src/auth/auth.service.ts`	**Session ID (affects all users):** `issueLoginSession` makes a new `sid`, puts it in the JWT, saves a `zuvy_user_sessions` row and returns `sessionId`. `refreshToken` and `generateTokensForSwitch` keep the `sid` of the current token. `logout` closes the session.	Session ID per login. All session writes are inside `try/catch`: a database problem is logged and never blocks login, refresh, org switch or logout. Responses only gain the extra `sessionId` field; nothing was removed or renamed.
`src/auth/auth.controller.ts`	Google login passes the client IP and browser (`user-agent`) to `login()`.	Stored on the session row.
`src/auth/strategies/jwt.strategy.ts`	`req.user[0].sessionId` = the token's `sid` (or `null` for tokens made before this change).	Lets any API read the current session ID.
`src/org/org.service.ts`	`switchOrg` passes the current access token to `generateTokensForSwitch`.	So the new tokens keep the same session ID.
`src/middleware/jwt.middleware.ts`	`GET /schools/states`, `GET /schools/cities` and `GET /schools` added to the public routes.	The dropdowns are needed before the student has an account.
`drizzle/schema.ts`	`zuvySchools` also has `village`, `management`, `classFrom`, `classTo`, and the index `(state, district, school_name)`.	Details that help a student recognise their school in the dropdown, and fast sorted lists.

File	Change	Why it was needed
`src/app.module.ts`	Registered `StudentAuthModule` and `SchoolsModule`.	Turns on the new endpoints.
`sample.env`	Added the new settings (Part C3).	Documentation for deployments.

Part C: Server setup for DevOps

Do these once per environment (dev first, then stage and production).

C1. Run the database migration

1. Connect to the environment's database with `psql`, DBeaver or pgAdmin.
2. Select the right schema. It is `main` unless the server runs with `ENV_NOTE=stage_template`:

   ```
   SET search_path TO main;            -- or: SET search_path TO stage_template;
   ```

3. Run these whole files, **in this order**:
   1. `drizzle/migrations/0043_create_zuvy_student_credentials.sql`
   2. `drizzle/migrations/0044_student_id_from_name.sql`
   3. `drizzle/migrations/0045_schools_and_sessions.sql`

   Skip the ones that already ran (on dev, 0043 and 0044 probably have). All three are safe to run twice.
4. Check the tables:

   ```
   -- 14 columns, the last four being state, city, school_name, udise_code
   SELECT column_name, data_type
   FROM information_schema.columns
   WHERE table_name = 'zuvy_student_credentials'
   ORDER BY ordinal_position;

   -- both tables should be listed
   SELECT table_name FROM information_schema.tables
   WHERE table_name IN ('zuvy_schools', 'zuvy_user_sessions');

   -- zuvy_schools must have village, management, class_from and class_to
   SELECT column_name FROM information_schema.columns
   WHERE table_name = 'zuvy_schools' ORDER BY ordinal_position;
   ```

> **Ran an earlier version of 0045?** Run the current file again. It adds the new `zuvy_schools` columns (`village`, `management`, `class_from`, `class_to`) and replaces the old index, without touching any data.

> **Run 0045 before deploying this version of the code.** Student ID sign-up and login read the new school columns, so without 0045 they fail with a 500 error. Google login keeps working without 0045 (the session row just isn't saved, and an error is logged), but the session IDs from that time are not recorded.

> Deploying the code before **0043/0044** is safe for existing users, because nothing else reads those tables. Only the `/auth/student/*` endpoints fail until they exist.

C2. Create the Cloudflare Turnstile widget (get the keys)

1. Log in at **dash.cloudflare.com** and open **Turnstile** in the left menu (on newer dashboards it's under *Application security*).
2. Click **Add widget**:
   - **Name:** `Zuvy Student Sign-up`
   - **Hostnames:** the **frontend** domains where students sign up, e.g. `dev.app.zuvy.org` and the production frontend domain. Not the API domains.
   - **Widget mode: Managed** (most students see one checkbox or nothing)
   - **Pre-clearance:** No
3. Click **Create**. Cloudflare shows two keys:
   - **Site key** → give it to the frontend team (it's public).
   - **Secret key** → put it **only** in the server `.env` (Part C3). Never in frontend code, chat or git.

Your domain does **not** need to use Cloudflare DNS or hosting. Turnstile works on any website and is free.

Use an organisation-owned Cloudflare account, not a personal one. If the person who owns a personal account leaves, nobody can rotate the secret or change the hostnames. If you must start with a personal account, invite a teammate as an admin (*Manage Account → Members*). The keys can be swapped later without any code change.

C3. Environment variables (server `.env`)

Variable	Required	Example / default	Meaning
`TURNSTILE_SECRET_KEY`	**Yes** (unless the captcha is disabled on dev)	From Cloudflare (C2)	Secret key used to check tokens. Without it (and without `CAPTCHA_DISABLED`), sign-up returns 503.
`TURNSTILE_ALLOWED_HOSTNAMES`	No	`dev.app.zuvy.org`	If set, the captcha must have been solved on one of these **frontend** sites (comma-separated, exact names). Leave empty when using Cloudflare test keys.
`CAPTCHA_DISABLED`	No	`true` on dev only	Skips the captcha check so testers can use Swagger. Must be lowercase `true`. **Ignored on production** (when `BASE_URL` contains `main-api`).
`SIGNUP_LIMIT_PER_MINUTE`	No	`60`	Sign-ups allowed per IP per minute
`SIGNUP_LIMIT_PER_HOUR`	No	`300`	Sign-ups allowed per IP per hour
`LOGIN_LIMIT_PER_15_MINUTES`	No	`600`	Student ID logins allowed per IP per 15 minutes
`TRUST_PROXY`	No	`loopback,uniquelocal`	Proxies trusted for the client IP. The default fits nginx on the same EC2 host in front of Docker.

How to write them:

```
TURNSTILE_SECRET_KEY="0x4AAAAAAA...your-secret..."
TURNSTILE_ALLOWED_HOSTNAMES="dev.app.zuvy.org"
```

- Double quotes are fine (they match the rest of the file). Use **no spaces** around `=` and no trailing spaces.
- Docker Compose reads `.env` **only when the container starts**. After editing, recreate the container:

  ```
  docker compose -f compose.yaml up -d --force-recreate             # dev
  docker compose -f compose_production.yml up -d --force-recreate   # production
  ```

- Check the container sees the key without printing it (prints only its length; `0` means missing):

  ```
  docker exec zuvy_container sh -c 'echo ${#TURNSTILE_SECRET_KEY}'
  ```

Cloudflare test keys (for dev and testing; they never show a real puzzle):

Purpose	Value
Backend test secret: always passes	`1x0000000000000000000000000000000AA`
Backend test secret: always fails	`2x0000000000000000000000000000000AA`
Frontend test site key: always passes	`1x00000000000000000000AA`
Token produced by test site keys	`XXXX.DUMMY.TOKEN.XXXX`

C4. nginx: pass the student's real IP to the API

The API runs in Docker behind nginx. For the per-IP limits to see each student's own IP (and not nginx's), nginx must forward it. On Ubuntu, `include proxy_params;` already sets the needed headers (`Host`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto`), so don't add them a second time.

Recommended API server block for dev:

```
server {
    server_name dev.api.zuvy.org;

    location / {
        proxy_pass http://127.0.0.1:5001/;
        include proxy_params;      # sets Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/dev.api.zuvy.org/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/dev.api.zuvy.org/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot
}
```

- Use `127.0.0.1` rather than `localhost` in `proxy_pass` (`localhost` can resolve to IPv6 first).
- Apply with `sudo nginx -t && sudo systemctl reload nginx`.
- If `nginx -t` warns `conflicting server name`, another file also declares the same domain. Find it with `sudo grep -rn "dev.api.zuvy.org" /etc/nginx/sites-enabled/ /etc/nginx/conf.d/` and remove the duplicate

(often a backup copy left in `sites-enabled`).

- If **Cloudflare's proxy** sits in front of nginx (`curl -sI https://dev.api.zuvy.org | grep -i cf-ray` prints something), nginx also needs its `real_ip` module with Cloudflare's IP ranges and `real_ip_header CF-Connecting-IP;`. Otherwise every request carries Cloudflare's IP.

C5. Docker: only nginx should reach the container

The compose files publish the API port on **all** network interfaces (`5001` dev, `4778` stage, `4777` production), so someone could skip nginx and send a fake IP header. Because nginx uses `127.0.0.1`, bind the port to localhost in the compose file:

```
ports:
  - '127.0.0.1:5001:5000'
```

Then recreate the container. If you prefer not to change the compose files, block these ports from the internet in the **EC2 security group** instead. (Change the compose files in git, not only on the server, because the deploy runs `git pull`.)

C6. Outbound access to Cloudflare

The backend calls `https://challenges.cloudflare.com` to check tokens. EC2 allows outbound traffic by default; confirm from inside the container:

```
docker exec zuvy_container wget -qO- --spider https://challenges.cloudflare.com && echo OK
```

C7. Server checklist

- [ ] Migrations `0043`, `0044` and `0045` run on this environment's database, **0045 before the deploy** (C1)
- [ ] School list loaded with `scripts/import-schools.ts --yes`; `GET /schools/states` returns 35 states (C8)
- [ ] Turnstile widget created; site key given to frontend; secret key in `.env` (C2, C3)
- [ ] `.env` values added and container recreated; key length check is not `0` (C3)
- [ ] nginx uses `include proxy_params;` and `127.0.0.1`; `nginx -t` shows no conflict warning (C4)
- [ ] Container port bound to `127.0.0.1` or blocked in the security group (C5)
- [ ] Container can reach `challenges.cloudflare.com` (C6)
- [ ] QA test TC-42 passes: one network blocked by the limit, another network still works (Part G)

C8. Load the school list (one command, once per environment)

The sign-up dropdowns show only what is in `zuvy_schools`. **Until the list is loaded, the dropdowns are empty and nobody can sign up with a Student ID.** Loading it is a single command, run by a developer or DevOps once per environment (dev, then production) after migration 0045.

1. Download the file (once; about 400 MB):

1. Open the India Data Portal and find the dataset "**UDISE**" (search "UDISE" on indiadataportal.com).
2. Download the file "**Basic Details of Schools**" as **CSV**. It is named `basic-details-of-schools.csv`.
3. **Don't open or save it in Excel.** Excel can't show that many rows and changes the UDISE codes. Use the file exactly as downloaded.

2. Check the file (writes nothing). From the project folder, on a machine whose `.env` points at the target database:

```
npx ts-node --transpile-only scripts/import-schools.ts "/path/to/basic-details-of-schools.csv"
```

It reads the whole file (about 20 seconds) and prints what it would load. With the October 2026 download it prints:

```
Schools in the file:     13,72,195
Good schools:            13,72,195
Skipped (bad rows):      0
Leading zero added back: 4,43,814 codes
Schools per state:
  Andaman and Nicobar Islands    400
  Andhra Pradesh                 53,622
  ...
Checked in 17 s. To load it into database "dev" on <host>, schema "main", run again with --yes
```

Read the last line carefully: it names the database and schema the data will go into (from `DB_HOST`, `DB_NAME` and `ENV_NOTE` in `.env`).

3. Load it by adding `--yes`:

```
npx ts-node --transpile-only scripts/import-schools.ts "/path/to/basic-details-of-schools.csv" --yes
```

- It saves 1,000 schools per statement and prints progress every 50,000 schools. Expect roughly **5–15 minutes** for all of India; it is fastest when run on the server, close to the database.
- The list needs about **430 MB** of database space (including indexes).
- It's **safe to stop and run again**, and safe to run again next year with a newer file: schools are matched by UDISE code, so they're updated, never duplicated. Nothing is ever deleted.
- To load only some states: `--states "Telangana,Bihar"` (any letter case).
- On the server, inside the container: copy the file in with `docker cp basic-details-of-schools.csv zuvy_container:/tmp/`, then run `docker exec -it zuvy_container npx ts-node --transpile-only scripts/import-schools.ts /tmp/basic-details-of-schools.csv --yes`. Delete `/tmp/basic-details-of-schools.csv` afterwards.

4. Check: `GET /schools/states` returns 35 states. The API keeps the state and city lists for up to 10 minutes, so new states can take that long to appear; restarting the container shows them at once.

What the import does with the file's data:

- Adds back the leading zero that the file dropped from 4,43,814 codes (states 01–09: J&K, Himachal, Punjab, Chandigarh, Uttarakhand, Haryana, Delhi, Rajasthan, UP).
- Keeps the 216 official codes that contain letters (New Delhi, UP, Meghalaya), e.g. `070501ND201`.
- Puts school names that break over two lines on one line, stores pincode `0` as blank, and skips any school whose `status` isn't "Operational" (all are, in this file).

C9. Add or correct a school (super admin, rarely needed)

If a student says their school is missing (for example a school that opened after the UDISE file was made), a super admin adds it with a small CSV.

1. Prepare the CSV (Excel or Google Sheets, saved as "CSV UTF-8"; format the UDISE code column as **Text** so leading zeros stay):

Column	Required	Rules	Other accepted header names
`udise_code`	Yes	11 characters (digits; a dropped leading zero is added back)	`udise`, `udisecode`, `udise_sch_code`, `udise_school_code`
`school_name`	Yes	Up to 255 characters	`schoolname`, `school`
`state`	Yes	Use the spelling shown in the state dropdown; capitals don't matter	`state_name`
`district`	Yes	Use the spelling shown in the city dropdown; capitals don't matter	`district_name`, `city`
`block`, `village`, `pincode`, `management`, `class_from`, `class_to`	No	Optional details; a value that doesn't fit is left blank	`subdistrict_name`, `village_name`, `pin_code`, `pin`

Header names are not case-sensitive, and spaces or hyphens count as `_` ("UDISE Code" works). Columns can be in any order; extra columns are ignored. Rows copied from the UDISE file work as they are. Example:

```
udise_code,school_name,state,district,village,pincode
36070799999,ZPHS NEW COLONY,Telangana,Hyderabad,Madhapur,500081
```

A state or city that already exists is saved with the list's spelling ("HYDERABAD" is saved as "Hyderabad"), so no second city appears in the dropdown. A misspelt city ("Hydrabad") would become a new city, so copy the spelling from the dropdown.

2. Upload it: in Swagger, authorize with a super admin token (Part E4), open `POST /schools/admin/upload`, click **Try it out**, choose the file and **Execute**. The file must end in `.csv` and be at most 25 MB. New states and cities appear in the dropdowns straight away.

3. Read the result:


```
{
  "status": "success",
  "message": "2 schools added, 1 updated, 1 rows skipped",
  "data": {
    "totalRows": 4,
    "inserted": 2,
    "updated": 1,
    "skipped": 1,
    "errors": [
      {
        "row": 4,
        "reason": "UDISE code must be 11 digits or capital letters (got \"12345\")"
      }
    ],
    "moreErrors": 0
  }
}
```


- **Added:** UDISE codes that were not in the list yet.

- **Updated:** UDISE codes already in the list. Their details are replaced with the CSV's values, and the school is made active again. Use this to correct a name.
- **Skipped:** rows with a problem. `row` is the row number in the file, counting the header as row 1. Up to 100 errors are listed; `moreErrors` counts the rest. Fix those rows and upload the file again: rows that were already saved are simply updated.
- Uploading never deletes a school. To hide a school from the dropdowns, mark it inactive in the database (Part H).

4. Check: search for the school in `GET /schools?state=...&city=...&search=...`.

Part D: Frontend guide

D1. What the frontend needs to build

Screen	What it does
Student sign-up page	Name, password, confirm password, **State** → **City** → **School** dropdowns, the **Cloudflare box**, and a "Create account" button
Student ID card	Shown right after sign-up: the Student ID in big letters, with **Download / Print** buttons
"Login with Student ID"	Next to "Sign in with Google": Student ID + password fields

Everything after login (dashboard, courses, refresh, logout) is the **same as for Google users**. Login responses (Google and Student ID) now also include a `sessionId` (Part F9); the frontend doesn't need to do anything with it.

The school dropdowns (Part F7 has the full API):

1. **State:** on page load, `GET /schools/states` → 35 states, like `["Andaman and Nicobar Islands", "Andhra Pradesh", ...]`. Show them exactly as received.
2. **City:** when a state is picked, `GET /schools/cities?state=Telangana` → `["Adilabad", ..., "Hyderabad", ...]` (up to 74 districts in a state). Clear the city and school.
3. **School:** when a city is picked, load `GET /schools?state=Telangana&city=Hyderabad`. A city can have **thousands of schools** (Jaipur has 8,906), so make this a **search box with a list under it**, not a plain dropdown:
   - As the student types, call it again with `&search=<text>`, about 300 ms after the last key press.
   - The student can type **several words** in any order and case: part of the school name, the **village**, the **block**, the **pincode** or the start of the **UDISE code** ("govt rampur", "sarvodaya malcha", "500081"). Every word must match.
   - Show 50 at a time (`limit`, `offset`); `total` says how many match. When there are many, show "Type your school's name or village to find it faster".
4. Many schools have similar names, so **show each school on two lines** using the extra fields:

   ```
   GOVT MIDDLE SCHOOL DUGONG CREEK
   Ramakrishnapur, Brc-Little Andaman · Classes 1–8 · 35010300103
   ```

   Line 1: `schoolName`. Line 2: `village`, `block` (skip blanks) · "Classes `classFrom`–`classTo`" (show `0` as "Pre-primary") · `udiseCode`. Showing `management` too (for example "Govt" for "Department of Education",

"Private" for "Private Unaided (Recognized)") helps students tell similar schools apart.

5. On submit, send the picked `state`, `city` and the school's `udiseCode`. Don't send the school name; the backend takes it from the list.
6. These three APIs need **no token**.

If the student's school is missing from the list, they can't finish sign-up. Show a short note under the list, such as "School not in the list? Ask your teacher to contact the Zuvy team.". A super admin then adds it (Part C9). With 13.7 lakh schools loaded this should be rare.

D2. How the Cloudflare token works in the frontend

```
Sign-up page loads
  └─► Cloudflare box appears (uses the SITE KEY) and checks the browser
        └─► Cloudflare gives the page a TOKEN (via the onSuccess callback)
              └─► Enable the "Create account" button (once a school is picked too)
                    └─► POST /auth/student/signup
                          { name, password, state, city, udiseCode, captchaToken: TOKEN }
                          ├─► success (201): save session, show the Student ID card
                          └─► error: show the message AND reset the box (to get a new token)
```

Rules:

1. **One token = one sign-up attempt.** After any attempt (success or error), reset the box to get a fresh token.
2. **Tokens expire after 5 minutes.** If the student waits too long, Cloudflare calls `onExpire`; clear the token and keep the button disabled until a new one arrives.
3. **Only send the token to `/auth/student/signup`.** Login doesn't use a captcha.
4. **Don't store the token** (no localStorage, no cookies). It's only for that one request.
5. **Never put the secret key in the frontend**, and never call Cloudflare's `siteverify` URL from the browser. The backend does that.

D3. Site key per environment

Environment	Site key to use	Backend secret it must pair with
Local development	`1x00000000000000000000AA` (Cloudflare test key, always passes)	Test secret `1x0000000000000000000000000000000AA`, or `CAPTCHA_DISABLED=true`
Dev (`dev.app.zuvy.org`)	Real site key from Cloudflare (or the test key while testing)	Matching real secret (or the test secret)
Production	Real site key	Real secret

Keep it in the frontend's env config, for example `NEXT_PUBLIC_TURNSTILE_SITE_KEY` (Next.js) or `VITE_TURNSTILE_SITE_KEY` (Vite). It's public, so this is safe.

> A token made with a **test** site key is rejected by a **real** secret key. Keep the frontend and backend keys from the same pair.

D4. Code example: React / Next.js

Install the small wrapper library:

```
npm install @marsidev/react-turnstile
```

Sign-up form:

```
'use client';
import { useEffect, useRef, useState } from 'react';
import { Turnstile, type TurnstileInstance } from '@marsidev/react-turnstile';

const API_URL = process.env.NEXT_PUBLIC_API_URL; // e.g. https://dev.api.zuvy.org
const SITE_KEY = process.env.NEXT_PUBLIC_TURNSTILE_SITE_KEY!;

export function StudentSignupForm() {
  const turnstileRef = useRef<TurnstileInstance>(null);
  const [captchaToken, setCaptchaToken] = useState('');
  const [school, setSchool] = useState<PickedSchool | null>(null);
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState('');

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    if (!school) return;
    const form = new FormData(e.currentTarget);
    setSubmitting(true);
    setError('');
    try {
      const res = await fetch(`${API_URL}/auth/student/signup`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          name: form.get('name'),
          password: form.get('password'),
          state: school.state,
          city: school.city,
          udiseCode: school.udiseCode,
          captchaToken,
        }),
      });
      const data = await res.json();
      if (!res.ok) {
        setError(signupErrorMessage(res.status, data.message));
        return;
      }
      saveSession(data); // the same function used after Google login
      showStudentIdCard(data.studentId); // big card with Download / Print
    } catch {
      setError('Network problem. Please check your internet and try again.');
    } finally {
      // A token works only once: always get a fresh one for the next attempt.
      turnstileRef.current?.reset();
      setCaptchaToken('');
      setSubmitting(false);
```

```
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="name"
        placeholder="Your full name"
        minLength={2}
        maxLength={100}
        required
      />
      <input
        name="password"
        type="password"
        placeholder="Password"
        minLength={6}
        maxLength={64}
        required
      />
      <SchoolPicker onChange={setSchool} />
      <Turnstile
        ref={turnstileRef}
        siteKey={SITE_KEY}
        onSuccess={setCaptchaToken}
        onExpire={() => setCaptchaToken('')}
        onError={() => setCaptchaToken('')}
      />
      {error && <p role="alert">{error}</p>}
      <button type="submit" disabled={!captchaToken || !school || submitting}>
        {submitting ? 'Creating account…' : 'Create account'}
      </button>
    </form>
  );
}

type PickedSchool = { state: string; city: string; udiseCode: string };
type SchoolOption = {
  udiseCode: string;
  schoolName: string;
  village: string | null;
  block: string | null;
  pincode: string | null;
  management: string | null;
  classFrom: number | null;
  classTo: number | null;
};
```

```
const getJson = (path: string) =>
  fetch(`${API_URL}${path}`).then((r) => r.json());

// Second line under the school name, e.g.
// "Ramakrishnapur, Brc-Little Andaman · Classes 1–8 · 35010300103"
function schoolDetails(s: SchoolOption) {
  const place = [s.village, s.block].filter(Boolean).join(', ');
  const classFrom = s.classFrom === 0 ? 'Pre-primary' : s.classFrom;
  const classes =
    s.classFrom != null && s.classTo != null
      ? `Classes ${classFrom}–${s.classTo}`
      : '';
  return [place, classes, s.udiseCode].filter(Boolean).join(' · ');
}

// State -> city -> school. Use your design system's components;
// the parts that matter are the API calls, the resets and the two lines.
function SchoolPicker({
  onChange,
}: {
  onChange: (s: PickedSchool | null) => void;
}) {
  const [states, setStates] = useState<string[]>([]);
  const [cities, setCities] = useState<string[]>([]);
  const [schools, setSchools] = useState<SchoolOption[]>([]);
  const [total, setTotal] = useState(0);
  const [state, setState] = useState('');
  const [city, setCity] = useState('');
  const [search, setSearch] = useState('');
  const [udiseCode, setUdiseCode] = useState('');

  useEffect(() => {
    getJson('/schools/states').then((r) => setStates(r.data ?? []));
  }, []);

  useEffect(() => {
    setCity('');
    setCities([]);
    setSearch('');
    onChange(null);
    if (state)
      getJson(`/schools/cities?${new URLSearchParams({ state })}`).then((r) =>
        setCities(r.data ?? []),
      );
  }, [state]);
```

```
  useEffect(() => {
    setSchools([]);
    setUdiseCode('');
    onChange(null);
    if (!state || !city) return;
    const timer = setTimeout(() => {
      const query = new URLSearchParams({ state, city, limit: '50' });
      if (search.trim()) query.set('search', search.trim());
      getJson(`/schools?${query}`).then((r) => {
        setSchools(r.data ?? []);
        setTotal(r.total ?? 0);
      });
    }, 300); // wait for the student to stop typing
    return () => clearTimeout(timer);
  }, [state, city, search]);

  return (
    <>
      <select value={state} onChange={(e) => setState(e.target.value)} required>
        <option value="">Select your state</option>
        {states.map((s) => (
          <option key={s}>{s}</option>
        ))}
      </select>
      <select
        value={city}
        onChange={(e) => setCity(e.target.value)}
        disabled={!state}
        required
      >
        <option value="">Select your city / district</option>
        {cities.map((c) => (
          <option key={c}>{c}</option>
        ))}
      </select>
      <input
        placeholder="Type your school's name, village or pincode"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
        disabled={!city}
      />
      {total > schools.length && (
        <small>
          Showing {schools.length} of {total}. Type your school's name or
          village to find it faster.
        </small>
      )}
```

```
      <div role="radiogroup" aria-label="Your school">
        {schools.map((s) => (
          <label key={s.udiseCode}>
            <input
              type="radio"
              name="school"
              value={s.udiseCode}
              checked={udiseCode === s.udiseCode}
              onChange={() => {
                setUdiseCode(s.udiseCode);
                onChange({ state, city, udiseCode: s.udiseCode });
              }}
            />
            <strong>{s.schoolName}</strong>
            <small>{schoolDetails(s)}</small>
          </label>
        ))}
      </div>
      <small>
        School not in the list? Ask your teacher to contact the Zuvy team.
      </small>
    </>
  );
}

function signupErrorMessage(status: number, message: string | string[]) {
  if (status === 400) return Array.isArray(message) ? message[0] : message; // validation or captcha
  if (status === 429) return message; // says how long to wait
  if (status === 503)
    return 'Sign-up is temporarily unavailable. Please try again later.';
  return 'Something went wrong. Please try again.';
}
```

Disabling the button while `submitting` also stops a double-click from creating two accounts.

If the server answers **400 "School not found"** or "**not in the selected state and city**", the list probably changed while the page was open: reload the dropdowns and ask the student to pick again. The captcha token is **not** used up by these two errors (the school is checked first), but resetting the box anyway, as the code above does, is simplest.

D5. Code example: plain HTML / JavaScript

```
<script
  src="https://challenges.cloudflare.com/turnstile/v0/api.js?onload=initCaptcha&render=explicit"
  async
  defer
></script>
<div id="captcha"></div>

<script>
  let captchaToken = '';
  let widgetId;

  function initCaptcha() {
    widgetId = turnstile.render('#captcha', {
      sitekey: 'YOUR_SITE_KEY',
      callback: (token) => {
        captchaToken = token;
      }, // enable the button here
      'expired-callback': () => {
        captchaToken = '';
      },
      'error-callback': () => {
        captchaToken = '';
      },
    });
  }

  // After every sign-up attempt (success or error):
  //   turnstile.reset(widgetId); captchaToken = '';
</script>
```

D6. Login with Student ID

```
const res = await fetch(`${API_URL}/auth/student/login`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ studentId, password }), // no captcha on login
});
```

- Accept the Student ID in any case; the backend ignores case and spaces around it.
- On success, handle the response **exactly like Google login**: store `access_token` and `refresh_token` the same way, then use the same refresh and logout calls.

D7. Error messages to show

API	Status	Meaning	What the UI should do
Sign-up	400	Invalid name/password, missing or failed captcha	Show the message; **reset the Cloudflare box**
Sign-up	400	`School not found...` or `...not in the selected state and city...`	Reload the dropdowns; ask the student to pick the school again
Sign-up	429	Too many sign-ups from this network	Show the server message (it says to wait a few minutes)
Sign-up	503	Captcha not configured on the server, or Cloudflare down	"Sign-up is temporarily unavailable. Please try again later."
Sign-up	500	Server or database problem	"Something went wrong. Please try again."
Login	401	Wrong Student ID or password	"Wrong Student ID or password."
Login	429	Account locked (5 wrong passwords), or too many logins	Show the server message (it says how many minutes to wait)

D8. After sign-up and login

- **Student ID card (very important):** there is no "forgot password" and no email, so the Student ID is the student's only way back in. Show it in big letters, for example "Your login is **riyasharmact1**", ask them to write it down (for example a "I have written it down" checkbox), and offer **Download** and **Print**.
- **Remember the last Student ID on the device** (localStorage) and pre-fill it on the login page. Never store the password.
- `user.email` **is** `null` for these students. Anywhere the UI shows the user's email (header, profile, leaderboard), show the name or the Student ID instead of "null".
- **Forgot password:** show a short note on the login page, e.g. "Forgot your password? Ask your teacher or the Zuvy team."

D9. Content-Security-Policy

If the frontend sets a CSP header, allow Cloudflare, or the box won't load:

```
script-src  ... https://challenges.cloudflare.com;
frame-src   ... https://challenges.cloudflare.com;
```

D10. Testing the frontend with Cloudflare test site keys

Test site key	Behaviour	Use it to test
`1x00000000000000000000AA`	Always passes (box visible)	The normal sign-up flow
`2x00000000000000000000AB`	Always fails (box visible)	No token arrives; button stays disabled; error shown
`3x00000000000000000000FF`	Always shows an interactive challenge	How the page looks when Cloudflare asks the student to click

D11. Frontend checklist

- [ ] Sign-up page with name, password, the State → City → School dropdowns and the Cloudflare box
- [ ] School list has a search box (several words: name, village, block, pincode or UDISE code), shows each school on two lines (name; village, block, classes, UDISE code), and resets when the state or city changes
- [ ] "School not in the list?" note under the dropdown
- [ ] "Create account" disabled until a school is picked and a token arrives, and while the request is running
- [ ] `state`, `city`, `udiseCode` and the token (`captchaToken`) sent in `POST /auth/student/signup`
- [ ] Box reset after every attempt; expired token clears the button
- [ ] Student ID card with Download / Print after sign-up
- [ ] "Login with Student ID" option next to Google, using `POST /auth/student/login`
- [ ] Session handled exactly like Google login (tokens, refresh, logout)
- [ ] No "null" shown where an email would appear
- [ ] Error messages from D7
- [ ] Site key from env config; secret key never in frontend code
- [ ] CSP allows `challenges.cloudflare.com` (if a CSP is used)
- [ ] Admin screen: "Add student by Student ID" (D12)

D12. Admin screen: add a student by Student ID

Org admins, ops and super admins need a way to put a Student ID student into a **private** course's batch. The existing "Add students" dialog (email list / CSV) can't do it, because it works by email.

What to build on the course's students page, next to the existing "Add students" button: an **"Add by Student ID"** dialog with:

1. A **Student ID** text field (accept any case; the backend lowercases it).
2. A **batch** dropdown (the course's batches; always send one).
3. An **Add** button that calls:

```
const res = await fetch(`${API_URL}/auth/student/admin/enroll`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${accessToken}`,
  },
  body: JSON.stringify({ studentId, bootcampId: course.id, batchId }),
});
```

Show the result clearly:

Status	Show
200	"Riya Sharma (riyasharmact1) added to Batch 2", using `data.name` from the response, so the admin can confirm it's the right student. Refresh the students list.
404	"No student with this Student ID" (or "Batch not found")
409	"This student is already in this course"
400	"This batch is full"

Status	Show
403	"You can only add students to your own organisation's courses. Switch to the right organisation."

Show the button only to **admin**, **ops** and **super admin** users. Hide it for instructors (they would get 403).

In the students list these students show **no email**. Show their name, and the Student ID if available.

Part E: Backend developer guide: testing in Swagger

E1. Run the unit tests locally

The project's `package.json` Jest config points to a `libs/` folder that doesn't exist, so `npm test` fails before running anything. Use this small local config instead (create it in the project root; **don't commit it**):

```
// jest.student-auth.config.js
module.exports = {
  rootDir: __dirname,
  roots: ['<rootDir>/src'],
  moduleFileExtensions: ['js', 'json', 'ts'],
  moduleDirectories: ['node_modules', '<rootDir>'],
  testEnvironment: 'node',
  transform: { '^.+\\.ts$': ['ts-jest', { isolatedModules: true }] },
};
```

```
npx jest -c jest.student-auth.config.js --runInBand --forceExit src/student-auth src/schools
```

Expected: `Test Suites: 7 passed`, `Tests: 87 passed`. (Error logs such as `Turnstile request failed: timeout` are expected: the tests exercise failure paths.)

E2. Swagger basics

1. Open **`https://dev.api.zuvy.org/apis`** (locally: `http://localhost:<PORT>/apis`). Swagger is turned off on production.
2. Find the sections "**Student ID Authentication**" (5 endpoints) and "**Schools**" (4 endpoints).
3. **To call an endpoint:** expand it, click **Try it out**, edit the body, click **Execute**.
4. **To call a protected endpoint:** copy the `access_token` from a sign-up or login response → click **Authorize** (padlock, top right) → paste **only the token** (Swagger adds `Bearer`) → **Authorize** → **Close**.
5. **To switch user: Authorize** → **Logout**, then authorize with the other token.

E3. The captcha in Swagger

Swagger can't show the Cloudflare box, so on the dev server use one of these:

Dev `.env` setting	What to send as `captchaToken` in sign-up
`CAPTCHA_DISABLED="true"`	Leave `captchaToken` out (or send anything)
`TURNSTILE_SECRET_KEY="1x0000000000000000000000000000000AA"`	`"XXXX.DUMMY.TOKEN.XXXX"` (runs the real check against Cloudflare)

On production neither works: a real token from the frontend is always required.

E4. Getting a super admin token (for the admin APIs)

1. Log in to the **dev web app** (`https://dev.app.zuvy.org`) with a Google account that is a super admin on dev.
2. Open DevTools (F12) → **Network** tab.
3. Click any request to the API and copy the value after `Authorization: Bearer `.
4. Paste it in Swagger's **Authorize** dialog.

No super admin account on dev? Ask an existing super admin to add your email with `POST /super-admin`.

E5. Swagger walkthrough (about 20 minutes)

Use `CAPTCHA_DISABLED="true"` on dev for steps 1–13. The `TC-xx` numbers point to the full test cases in Part G3.

0. **School list** (G3.0): make sure the UDISE list is loaded on dev (Part C8), try the three dropdown APIs **without** a token (TC-57 to TC-60, TC-70, TC-71), then the upload as a super admin (TC-53 to TC-56, TC-72).
1. **Sign up** a student (TC-01) with `state`, `city` and `udiseCode` from step 0. Check: status 201, `studentId` like `riyasharmact1`, `school` filled in, a `sessionId`, `user.email` is `null`, `user.rolesList` is `["student"]`. Note the `studentId`, `access_token`, `refresh_token` and `sessionId`. Then try a wrong school (TC-61 to TC-64).
2. **Check validation** (TC-03 to TC-05): short name, short password and an extra `email` field each return 400.
3. **Log in** with the Student ID (TC-06) and with it in lowercase (TC-07). Both 200; `showTooltip` is `false`.
4. **Wrong password and unknown ID** (TC-08, TC-09): both 401 with the **same** message.
5. **Use the student token** (TC-13 to TC-15): authorize with the `access_token`; `GET /student/` returns 200 (not 401), and opening a public course enrols the student.
6. **Refresh and logout** (TC-17 to TC-19), and the **session ID** (TC-65 to TC-69): same `sessionId` after refresh, new one after a new login, `ended_at` set after logout.
7. **Lockout** (TC-10, TC-11): 5 wrong passwords, then the correct one returns 429.
8. **Super admin tools** (TC-20 to TC-32): authorize as a super admin; reset a password (generated and chosen), search by name, enrol in a private course with a `batchId`, try the edge cases.
9. **Non-admin access** (TC-31, TC-32): the admin APIs return 403 for a normal user and 401 with no token.
10. **Enrolment by org admin and ops** (TC-47 to TC-52): log in to the dev web app as an **org admin** and as an **ops** user, copy their tokens, and enrol into their own organisation's course (200) and another organisation's course (403). Check `GET /trackinglog` shows the "enroll_student" entry.
11. **Database check** (Part H): the new rows exist, `email` is NULL, `password_hash` starts with `scrypt$`.
12. **Response headers:** in a sign-up or login response, check the `x-ratelimit-remaining-...` headers are present (they show the per-IP limits are active).
13. **Logs:** run `docker logs zuvy_container --tail 100`. A password reset logs `Password reset for Student ID ...`; a rejected captcha logs `Captcha rejected: ...`.
14. **Captcha and limits** (TC-37 to TC-46): needs `.env` changes on dev; coordinate with DevOps and restore the values afterwards.

E6. Before merging

- [ ] Unit tests pass (E1)
- [ ] Type check shows no new errors: `npx tsc --noEmit -p tsconfig.json`
- [ ] Swagger walkthrough E5 steps 1–13 pass on dev
- [ ] Regression tests RG-01 to RG-12 (Part G4) pass on dev

Part F: API reference

All examples use `riyasharmact1` as the Student ID. Use the ID you actually receive.

F1. Sign up: `POST /auth/student/signup`

Creates the account and logs the student in straight away.

- **Login needed:** no (public)
- **Request body:**

```
{
  "name": "Riya Sharma",
  "password": "riya@2026",
  "state": "Andaman and Nicobar Islands",
  "city": "Andamans",
  "udiseCode": "35010300101",
  "captchaToken": "XXXX.DUMMY.TOKEN.XXXX"
}
```

- **Rules:**
  - `name`: 2–100 characters. Spaces at the start and end are removed, and repeated spaces become one.
  - `password`: 6–64 characters.
  - `state`: required, from `GET /schools/states`. Any letter case.
  - `city`: required, the district from `GET /schools/cities`. Any letter case.
  - `udiseCode`: required, the 11-character code from `GET /schools` (digits; a few official codes contain letters, any case). Must be an **active** school in our list, **in that state and city**. The school name is not sent: it is taken from the list.
  - `captchaToken`: the token from the Cloudflare box. Required unless `CAPTCHA_DISABLED=true` on dev.
  - No other fields are allowed. For example, sending `email` or `schoolName` returns 400.
- **Order of checks:** per-IP limit → input validation → school check → captcha → account creation. A wrong school or a failed captcha never creates anything in the database, and a wrong school doesn't use up the captcha token.
- **Success:** `201 Created`

```
{
  "studentId": "riyasharmact1",
  "school": {
    "state": "Andaman and Nicobar Islands",
    "city": "Andamans",
    "schoolName": "GOVT SENIOR SECONDARY SCHOOL RAMAKRISHNAPUR",
    "udiseCode": "35010300101"
  },
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "sessionId": "6f1c2b8e-4d7a-4b9e-9c1a-2f3d4e5a6b7c",
  "showTooltip": true,
  "user": {
    "id": "48213",
    "email": null,
    "name": "Riya Sharma",
    "profilePicture": null,
    "role": "student",
    "center": null,
    "rolesList": ["student"],
    "orgId": null,
    "orgName": null,
    "isPoc": false,
    "permissions": {}
  }
}
```


Everything except `studentId` and `school` has exactly the same shape as the Google login response (`POST /auth/login`). `school` uses the school list's spelling, whatever letter case was sent.

- **Errors:**

Status	When	Example message
400	Name too short or long, password too short or long, or an extra field	`["password must be longer than or equal to 6 characters"]`
400	`state`, `city` or `udiseCode` missing, or `udiseCode` not 11 characters	`["udiseCode must be the 11-character UDISE code of the school"]`
400	UDISE code not in the school list, or the school is inactive	`School not found. Please pick your school from the list.`
400	The school is in a different state or city than the ones sent	`This school is not in the selected state and city. Please pick it again.`
400	No captcha token	`Captcha is required`
400	Cloudflare rejected the token (invalid, expired, already used, or solved on another site)	`Captcha verification failed. Please try again.`

Status	When	Example message
429	Too many sign-ups from this IP	`Too many requests from this network. Please wait a few minutes and try again.`
503	`TURNSTILE_SECRET_KEY` not set on the server	`Sign-up is temporarily unavailable. Please try again later.`
503	Cloudflare couldn't be reached	`Could not verify the captcha. Please try again.`
500	Table or column missing (migration 0043, 0044 or 0045 not run) or a database error	`Could not create the account` / `Internal server error`

F2. Log in: `POST /auth/student/login`

- **Login needed:** no (public)
- **Request body:**

```
{
  "studentId": "riyasharmact1",
  "password": "riya@2026"
}
```

  `studentId` is case-insensitive and spaces around it are ignored. There is no captcha on login.

- **Success:** `200 OK`: same shape as sign-up (`studentId`, `school`, `access_token`, `refresh_token`, `sessionId`, `showTooltip`, `user`). `showTooltip` is `false` after the first login. `sessionId` is **new on every login**. `school` is `null` for accounts created before the school fields existed.
- **Errors:**

Status	When	Message
400	Field missing or empty	validation message
401	Wrong Student ID **or** wrong password (deliberately the same message)	`Invalid Student ID or password`
429	5 wrong passwords in a row; this account is locked for 15 minutes	`Too many wrong attempts. Try again after 15 minute(s).`
429	Too many login attempts from this IP (default 600 per 15 minutes)	`Too many requests from this network. Please wait a few minutes and try again.`

F3. Reset a password (super admin): `POST /auth/student/admin/reset-password`

Use this when a student forgets their password.

- **Login needed:** yes, as a **super admin**
- **Request body: set a specific password**

```
{
  "studentId": "riyasharmact1",
  "newPassword": "newpass123"
}
```

- **Request body: let the server generate one** (8 easy-to-read characters)

```
{
  "studentId": "riyasharmact1"
}
```

- **Success:** `200 OK`

```
{
  "status": "success",
  "message": "Password updated. Share it with the student; it is not shown again.",
  "data": {
    "studentId": "riyasharmact1",
    "userId": "48213",
    "name": "Riya Sharma",
    "password": "k7mq4x2d"
  }
}
```

  The reset also **removes any lockout**. Who reset it and when is saved in `last_reset_by` and `last_reset_at`.

- **Errors:**

Status	When	Message
400	`newPassword` shorter than 6 or longer than 64 characters	validation message
401	No token, or an invalid token	`Token not found` / `Invalid token`
403	Caller is not a super admin	`Only super admins can manage Student IDs`
404	Student ID does not exist	`Student ID not found`

F4. Find students (super admin): `GET /auth/student/admin/students`

Use this to look up a student's Student ID by name, or to check an account.

- **Login needed:** yes, as a **super admin**
- **Query parameters** (pass `studentId` or `name`, or both):

Param	Example	Meaning
`studentId`	`riyasharmact1`	Exact Student ID
`name`	`riya`	Part of the name (case-insensitive)
`limit`	`20`	1–100, default 20

- **Example:** `GET /auth/student/admin/students?name=riya`

- **Success:** `200 OK`

```
{
  "status": "success",
  "data": [
    {
      "userId": "48213",
      "studentId": "riyasharmact1",
      "name": "Riya Sharma",
      "state": "Andaman and Nicobar Islands",
      "city": "Andamans",
      "schoolName": "GOVT SENIOR SECONDARY SCHOOL RAMAKRISHNAPUR",
      "udiseCode": "35010300101",
      "createdAt": "2026-10-08 10:15:22.481+00",
      "lastLoginAt": "2026-10-08 10:20:03.112+00",
      "lockedUntil": null,
      "lastResetAt": null
    }
  ]
}
```

- **Errors:** 400 (neither `studentId` nor `name` given: `Pass a studentId or a name to search`), 401, 403.

F5. Enrol in a course (super admin, or org admin / ops): `POST /auth/student/admin/enroll`

The existing admin "add students" API (`POST /bootcamp/students/:bootcamp_id`) finds students **by email**, so it can't add these students. Use this endpoint for **private** courses. Public courses don't need it.

- **Who can call it:**

Caller	Can enrol into
Super admin	Any course
Admin or **ops** of an organisation	Only courses of the organisation they are **currently logged into** (their active org)
Instructor, student, anyone else	Nothing (403)

  An admin or ops user who belongs to several organisations must first switch to the course's organisation (`POST /org/switch-org`), then call this API with the new token.

- **Request body:**

```
{
  "studentId": "riyasharmact1",
  "bootcampId": 12,
  "batchId": 34
}
```

  `batchId` is optional, but **always send it**. `GET /student/` (the student's "My courses") only lists enrolments that have a batch.

- **Order of checks:** course looked up → permission checked (403) → course exists (404) → Student ID exists (404) → batch belongs to the course and has space → enrol. The permission is checked first, so a caller without permission can't learn whether a course or Student ID exists.
- **Success:** `200 OK`. The response includes the student's **name**, so the admin can confirm they typed the right Student ID:

```
{
  "status": "success",
  "message": "Student riyasharmact1 enrolled in Full Stack Web Development",
  "data": {
    "studentId": "riyasharmact1",
    "userId": "48213",
    "name": "Riya Sharma",
    "bootcampId": 12,
    "bootcampName": "Full Stack Web Development",
    "batchId": 34
  }
}
```

  If the student is already in the course and you pass a **different** `batchId`, the student is moved: `"message": "Student moved to the selected batch"`, with the same `data`.
- **Activity log:** every call is recorded in the organisation's activity log (`zuvy_tracking_logs`, shown by `GET /trackinglog`) as "**enroll_student**", exactly like the normal "add students" API, including failed attempts.
- **Errors:**

Status	When	Message
400	Batch is full	`The maximum capacity for the batch has been reached`
403	Not a super admin, and not admin/ops of the course's organisation	`Only super admins, or admins and ops of this course's organisation, can enrol students by Student ID`
404	Wrong Student ID, course or batch	`Student ID not found` / `Course not found` / `Batch not found in this course`
409	Already enrolled (same batch, or no batch given)	`Student is already enrolled in this course`

F6. Existing endpoints that work unchanged for these students

Endpoint	Body or note
`POST /auth/refresh`	`{ "refresh_token": "<refresh_token>" }`; returns a new pair (plus the **same** `sessionId`), and the old refresh token stops working
`POST /auth/logout`	Authorize with the student's access token; afterwards that token returns 401
`GET /student/`	Courses the student is enrolled in (`inProgressBootcamps`, `completedBootcamps`)
`GET /student/bootcamp/public`	Public courses

Endpoint	Body or note
`GET /tracking/allModulesForStudents/{bootcampId}`	Modules of a course (also auto-enrols in a public course)
`GET /tracking/bootcampProgress/{bootcampId}`	Course progress
`GET /student/leaderboard/{bootcampId}`	Leaderboard (the student appears with `email: null`)

F7. School dropdowns: `GET /schools/states`, `GET /schools/cities`, `GET /schools`

All three are **public** (no token) and show **active** schools only. Lists are sorted A to Z. States and cities can be sent in **any letter case** (`telangana`, `TELANGANA`); responses use the list's spelling.

States: `GET /schools/states`

```
{
  "status": "success",
  "data": [
    "Andaman and Nicobar Islands",
    "Andhra Pradesh",
    "...",
    "West Bengal"
  ]
}
```

Cities (districts) of a state: `GET /schools/cities?state=Telangana`

Param	Required	Rules
`state`	Yes	1–100 characters, any case

```
{
  "status": "success",
  "data": ["Adilabad", "...", "Hyderabad", "...", "Warangal"]
}
```

The state and city lists are kept in memory for 10 minutes, so these two calls are instant.

Schools of a city: `GET /schools?state=Andaman and Nicobar Islands&city=Andamans&search=dugong&limit=50&offset=0`

Param	Required	Rules
`state`	Yes	1–100 characters, any case
`city`	Yes	The district, 1–100 characters, any case
`search`	No	1–100 characters, up to 5 words, any order and case. **Every word** must appear in the school name, village or block, or be the start of the UDISE code or pincode
`limit`	No	1–200, default 50

Param	Required	Rules
`offset`	No	0 or more, default 0 (for "load more")


```
{
  "status": "success",
  "data": [
    {
      "udiseCode": "35010300103",
      "schoolName": "GOVT MIDDLE SCHOOL DUGONG CREEK",
      "village": "Ramakrishnapur",
      "block": "Brc-Little Andaman",
      "pincode": "744207",
      "management": "Department of Education",
      "classFrom": 1,
      "classTo": 8
    }
  ],
  "total": 1,
  "limit": 50,
  "offset": 0
}
```


Schools are sorted by name. `total` is the number of matching schools, so the UI knows whether to offer "load more". `village`, `block`, `pincode`, `management` and the classes can be `null` when the UDISE data doesn't have them (`classFrom` 0 means pre-primary). An unknown state or city returns an empty list, not an error. A missing `state` or `city`, or a `limit` above 200, returns **400**.

Search examples: `search=govt senior` (both words in the name), `search=rampur` (name or village), `search=744207` (pincode), `search=3501030` (UDISE code start).

F8. Add or correct schools (super admin): `POST /schools/admin/upload`

For a few missing or wrong schools. The full list is loaded once with the import command (Part C8), not with this API.

- **Login needed:** yes, as a **super admin**
- **Body:** `multipart/form-data` with one field, `file`: a `.csv` file of at most 25 MB. The columns and rules are in Part C9; the UDISE file's own column names also work.
- **What it does:** adds new UDISE codes, updates existing ones (all details; made active again), and skips bad rows. A state or city that already exists is saved with the list's spelling. It never deletes a school.
- **Success:** `200 OK`: see the example in Part C9 (`inserted`, `updated`, `skipped`, `errors` with row numbers, `moreErrors`).
- **Errors:**

Status	When	Message
400	No file, or the file name doesn't end in `.csv`	`A CSV file is required (field "file")` / `Upload a .csv file`

Status	When	Message
400	A required column is missing	`Missing column(s): district. Required: udise_code, school_name, state, district (or city).`
400	Only a header row, or a `"` that is never closed	`The CSV needs a header row and at least one school` / `The CSV has a quote (") that is never closed`
401	No token	`Token not found`
403	Not a super admin	`Only super admins can upload the school list`
413	File bigger than 25 MB	`File too large`

F9. Session ID (all users)

Every login gets a new session ID: Google login (`POST /auth/login`), Student ID sign-up and Student ID login.

Where	What you see
Login and sign-up responses	`"sessionId": "6f1c2b8e-..."` (new each time)
`POST /auth/refresh` response	`sessionId`: the **same** as before the refresh
`POST /org/switch-org` response	`sessionId`: the **same** as before the switch
Inside the JWT	Claim `sid`
In backend code	`req.user[0].sessionId` (`null` for tokens made before this change)
Database	`zuvy_user_sessions` (Part B3): `created_at` at login, `last_refreshed_at` on refresh, `ended_at` on logout

Tokens made **before** this change have no `sid`. Their first refresh (or org switch) starts a new session, recorded with `login_method` `token_refresh` (or `org_switch`); after that the ID stays the same.

A user who closes the browser without logging out keeps `ended_at` empty. To estimate when such a session stopped, use `last_refreshed_at`.

Part G: Tester guide

G1. What you need

- Access to the **dev web app** (`https://dev.app.zuvy.org`) and **dev Swagger** (`https://dev.api.zuvy.org/apis`).
- A **super admin** Google account on dev (for password resets and enrolment).
- An **org admin** and an **ops** Google account on dev, and the id of one course in **their** organisation and one course in **another** organisation (for TC-47 to TC-52).
- **Two different internet connections** for the rate-limit tests, for example office Wi-Fi and a phone hotspot.
- Someone who can change the dev server's `.env` (only for section G3.6).
- Optional: read access to the dev database (Part H).

Test in this order: **G2** (in the browser, once the frontend is ready), **G3** (APIs in Swagger), **G4** (regression for email users).

G2. Browser tests (after the frontend is ready)

ID	Steps	Expected result
UI-01	Open the student sign-up page	The Cloudflare box appears. "Create account" stays disabled until the box shows success.
UI-02	Enter a name and password, pick state, city and school, click "Create account"	Account created. A big Student ID card appears with Download / Print. The student is logged in.
UI-03	Download or print the Student ID card	The file or printout shows the correct Student ID.
UI-04	Log out, then "Login with Student ID" with the ID and password	Logged in; dashboard loads.
UI-05	Log in with the Student ID typed in capitals	Works.
UI-06	Log in with a wrong password	"Wrong Student ID or password." No hint about which one was wrong.
UI-07	Enter a wrong password 5 times, then the correct one	A message says the account is locked and how many minutes to wait.
UI-08	While UI-07's account is locked, log in to **another** Student ID account from the same Wi-Fi	Works: the lock only affects one account.
UI-09	Open the sign-up page, complete the box, wait **more than 5 minutes**, then click "Create account"	The button is disabled or an error appears; the box resets; trying again works.
UI-10	Double-click "Create account" quickly	Only **one** account is created (check with Part H).
UI-11	**School lab:** 10 or more devices on the **same Wi-Fi** sign up within a few minutes	**All succeed.** Nobody gets "Too many requests".
UI-12	As a Student ID student, open a public course, finish a chapter, a quiz and a coding problem	Works like a Google student. Progress and leaderboard update.
UI-13	Check every place that normally shows an email (header, profile, leaderboard)	No "null" or broken layout; the name or Student ID is shown instead.
UI-14	Close the browser and come back the next day	Still logged in (session refresh works like Google users).
UI-15	"Forgot password": a super admin resets it (TC-20), then log in with the new password	Works.
UI-16	On the same login page, sign in with **Google** as an email user	Works exactly as before.
UI-17	As an **org admin**, and again as **ops**, open a private course's students page and use **"Add by Student ID"** with a batch	The student's name is shown to confirm; the student appears in that batch and now sees the course.
UI-18	On the sign-up page, open the **State** dropdown, pick a state, then a city	States and cities are listed A to Z. Changing the state clears the city and school.
UI-19	Pick a city with many schools and type part of a school name in the school search	The list narrows as you type; each school shows its name and UDISE code.

ID	Steps	Expected result
UI-20	Try to click "Create account" without picking a school	Not possible (button disabled or the form asks for the school).
UI-21	Check the "School not in the list?" note under the school dropdown	It's shown and tells the student whom to ask.

G3. API tests in Swagger

Run **G3.0 first**: it loads the test schools that every sign-up needs. Write down the `studentId`, `access_token`, `refresh_token` and `sessionId` from TC-01; later tests use them. Sections G3.1–G3.5 assume the dev server has `CAPTCHA_DISABLED="true"`, so sign-up bodies can leave out `captchaToken`. If dev uses the Cloudflare test secret instead, add `"captchaToken": "XXXX.DUMMY.TOKEN.XXXX"` to every sign-up body. **Every** sign-up body in this guide (including G3.6) also needs the three school fields ("+ school", G3.1).

G3.0 School list (run this first)

First the real list. These tests need the UDISE list loaded on dev (Part C8); no token is needed.

ID	Steps	Expected result
TC-57	`GET /schools/states`	**200** with the 35 states and union territories, A to Z, in normal spelling (`Andaman and Nicobar Islands`, ...).
TC-58	`GET /schools/cities?state=RAJASTHAN`, then `?state=andaman and nicobar islands`	**200** both: 33 districts including `Jaipur`; then `["Andamans", "Nicobars", "North And Middle Andaman"]`. Any letter case works.
TC-59	`GET /schools?state=Rajasthan&city=Jaipur`, then the same with `&offset=50`	**200**: `total` is 8906, 50 schools sorted by name, each with `village`, `block`, `management`, `classFrom`, `classTo`. Codes start with `08` (leading zero added back). The next page continues A to Z.
TC-60	Same city with `&search=govt senior`, then `&search=<a pincode from TC-59>`, then `&search=<first 7 characters of a UDISE code>`	Every result has both words (in the name, village or block); then only schools with that pincode; then only schools whose code starts that way.
TC-70	`GET /schools?state=Delhi&city=New Delhi&search=070501nd201`	**200**: `GOVT. SARVODAYA VIDYALAYA, KITCHNER ROAD, ...` with `udiseCode` `070501ND201` (one of the few official codes with letters).
TC-71	`GET /schools?state=Delhi` (no city), and `GET /schools?state=Delhi&city=New Delhi&limit=500`	**400** both times (city missing; limit above 200).

Then the upload, with clearly fake test schools. Save this as `test-schools.csv`; rows 5 and 6 are wrong on purpose. On a shared dev database, tell the team first: "Test State" will appear in everyone's state dropdown until it's hidden afterwards (Part H).

```
udise_code,school_name,state,district,block,pincode
99000000001,Zuvy Test School Alpha,Test State,Test City,Test Block,500001
99000000002,Zuvy Test School Beta,test state,TEST CITY,,
99000000003,Zuvy Test School Gamma,Test State,Other Test City,,
12345,Zuvy Bad Code School,Test State,Test City,,
99000000001,Zuvy Duplicate Row,Test State,Test City,,
```

ID	Steps	Expected result
TC-53	As a super admin, `POST /schools/admin/upload` with `test-schools.csv`	**200** `3 schools added, 0 updated, 2 rows skipped`. `errors` lists row 5 (`UDISE code must be 11 digits or capital letters`) and row 6 (`Duplicate UDISE code 99000000001 (already on row 2)`).
TC-54	Upload the same file again	**200** `0 schools added, 3 updated, 2 rows skipped`. No duplicates are created.
TC-55	Upload as a **non-super admin** (student, admin, ops), and once with no token	**403** `Only super admins can upload the school list`; with no token **401**.
TC-56	Upload a `.txt` file, and a CSV without the `district` column	**400** `Upload a .csv file`; **400** `Missing column(s): district...`.
TC-72	Right after TC-53: `GET /schools/cities?state=TEST STATE`	**200** `["Other Test City", "Test City"]`: straight away, and only one "Test City" although row 3 wrote it as `TEST CITY` (the first spelling is kept).

G3.1 Sign-up and login

In this section "**+ school**" means adding these three fields to the sign-up body: `"state": "Andaman and Nicobar Islands", "city": "Andamans", "udiseCode": "35010300101"` (a real school from the list: GOVT SENIOR SECONDARY SCHOOL RAMAKRISHNAPUR).

ID	Steps	Expected result
TC-01	`POST /auth/student/signup` with `{"name": "Test Student One", "password": "test@123"}` + school	**201**. The response has `studentId` = `testonect1` (first + last name + number), `school` = `{"state": "Andaman and Nicobar Islands", "city": "Andamans", "schoolName": "GOVT SENIOR SECONDARY SCHOOL RAMAKRISHNAPUR", "udiseCode": "35010300101"}`, a `sessionId`, plus `access_token`, `refresh_token`, `user.email = null` and `user.rolesList = ["student"]`.
TC-02	Sign up again with the **same** name and password + school	**201** with `studentId` = `testonect2`: the same name gets the next number.
TC-03	Sign up with `{"name": "A", "password": "test@123"}` + school	**400**: the name is too short.
TC-04	Sign up with `{"name": "Test", "password": "123"}` + school	**400**: the password is too short.

ID	Steps	Expected result
TC-05	Sign up with `{"name": "Test", "password": "test@123", "email": "a@b.com"}` + school	**400**: `property email should not exist`.
TC-06	`POST /auth/student/login` with the Student ID from TC-01 and `test@123`	**200**. Same shape as TC-01 (including `school`), `showTooltip` is `false`, and the `sessionId` is **different** from TC-01's.
TC-07	Log in with the Student ID in **capitals** (`TESTONECT1`)	**200**.
TC-08	Log in with the right ID and a **wrong** password	**401** `Invalid Student ID or password`.
TC-09	Log in with an ID that doesn't exist (e.g. `nobody99`)	**401** with the **same** message as TC-08.
TC-61	Sign up with name and password but **no** `state` , `city` or `udiseCode`	**400** listing the missing fields.
TC-62	Sign up + school, but `"udiseCode": "99000000099"` (not in the list)	**400** `School not found. Please pick your school from the list.` No new account (Part H).
TC-63	Sign up + school, but `"city": "Nicobars"` (the school is in Andamans)	**400** `This school is not in the selected state and city. Please pick it again.`
TC-64	Sign up with `"state": "andaman and nicobar islands", "city": " ANDAMANS "` , `"udiseCode": "35010300103"`	**201**; `school.state` is `Andaman and Nicobar Islands` , `school.city` is `Andamans` and `school.schoolName` is `GOVT MIDDLE SCHOOL DUGONG CREEK` (the list's spelling is saved).
TC-73	Sign up with `"state": "delhi", "city": "new delhi"` , `"udiseCode": "070501nd201"`	**201**; `school.udiseCode` is `070501ND201` (codes with letters work in any case).

G3.2 Lockout

ID	Steps	Expected result
TC-10	Use the TC-02 account. Log in with a wrong password **5 times**.	Attempts 1–5 return **401**.
TC-11	Right after that, log in with the **correct** password	**429** `Too many wrong attempts. Try again after 15 minute(s).`
TC-12	Reset the password as a super admin (TC-20), then log in with the new password	**200**: the reset removed the lock.

G3.3 Using the token on normal student APIs

Authorize in Swagger with the student's `access_token` first.

ID	Steps	Expected result
TC-13	`GET /student/`	**200**, with `inProgressBootcamps` and `completedBootcamps` both empty for a new student. There must be **no 401**: this proves the middleware fix works.
TC-14	`GET /student/bootcamp/public` and pick a public course id, then call `GET /tracking/allModulesForStudents/{id}`	**200** with the modules.
TC-15	`GET /student/` again	The public course from TC-14 now appears in `inProgressBootcamps` (automatic enrolment).
TC-16	Open a chapter, mark it complete and submit a quiz or coding problem through the normal student APIs or the web app	Works the same as for a Google student. Progress and leaderboard update.
TC-17	`POST /auth/refresh` with `{"refresh_token": "<refresh_token>"}`	**200** with a new `access_token` and `refresh_token`.
TC-18	Call `POST /auth/refresh` again with the **old** refresh token	**401**.
TC-19	`POST /auth/logout` (authorized as the student), then `GET /student/` with the same token	Logout returns **201**; the next call returns **401**.

G3.4 Super admin tools

Authorize in Swagger with a **super admin** token first.

ID	Steps	Expected result
TC-20	`POST /auth/student/admin/reset-password` with `{"studentId": "<id from TC-02>"}`	**200** with a generated 8-character `password`.
TC-21	Log in as that student with the **old** password	**401**.
TC-22	Log in with the password from TC-20	**200**.
TC-23	Reset with `{"studentId": "<id>", "newPassword": "mychoice1"}`, then log in with `mychoice1`	Both **200**.
TC-24	Reset with `{"studentId": "nobody99"}`	**404** `Student ID not found`.
TC-25	`GET /auth/student/admin/students?name=test student`	**200**; both test accounts are listed.
TC-26	`GET /auth/student/admin/students` with no parameters	**400**.
TC-27	`POST /auth/student/admin/enroll` with a **private** course: `{"studentId": "<id>", "bootcampId": <id>, "batchId": <id>}`	**200** "enrolled in ...", with `data.name` = the student's name. As the student, `GET /student/` now lists the course.
TC-28	Repeat TC-27 exactly	**409** `Student is already enrolled in this course`.
TC-29	Repeat TC-27 with **another batch** of the same course	**200** `Student moved to the selected batch`.

ID	Steps	Expected result
TC-30	Enrol with a `batchId` from a different course	**404** `Batch not found in this course`.
TC-31	Authorize as a **normal student, admin, ops or instructor** (not a super admin) and call `reset-password` or `GET .../admin/students`	**403** `Only super admins can manage Student IDs`. (Enrolment rules for admins and ops are tested in G3.7.)
TC-32	Log out of Swagger (no token) and call any `/auth/student/admin/*` endpoint	**401**.

G3.5 Classes and attendance (needs a batch with a Student ID student in it)

ID	Steps	Expected result
TC-33	As an admin or instructor, create a **Google Meet** class for a batch that contains a Student ID student and email students	Class is created. Email students get the calendar invite.
TC-34	Create a **Zoom** class for the same batch	Class is created. Email students are invited.
TC-35	After the Zoom class ends and attendance is processed	Email students' attendance is recorded as before. The Student ID student is **not** matched automatically (see Part I).
TC-36	As an admin, mark the Student ID student present with `POST /bootcamp/attendance/mark`, body `[{"sessionId": <id>, "userId": <student user id>, "status": "present"}]`. Send this flat list, not the `{"attendance": [...]}` wrapper that Swagger suggests: the server reads `sessionId` from each item directly.	Attendance is saved for that student.

G3.6 Captcha and rate limits (needs changes to the dev server's `.env`)

Change `.env`, recreate the container (Part C3), and run the test. **Put the original values back afterwards.**

ID	`.env` on dev	Steps	Expected result
TC-37	`TURNSTILE_SECRET_KEY=1x0000000000000000000000000000000AA`, no `CAPTCHA_DISABLED`	Sign up **without** `captchaToken`	**400** `Captcha is required`
TC-38	Same as TC-37	Sign up with `"captchaToken": "XXXX.DUMMY.TOKEN.XXXX"`	**201**: the real Cloudflare check ran and passed
TC-39	`TURNSTILE_SECRET_KEY=2x0000000000000000000000000000000AA` (always fails)	Sign up with the dummy token	**400** `Captcha verification failed. Please try again.` No new row in `zuvy_student_credentials` (Part H).
TC-40	No `TURNSTILE_SECRET_KEY` and no `CAPTCHA_DISABLED`	Sign up	**503** `Sign-up is temporarily unavailable. Please try again later.`

ID	`.env` on dev	Steps	Expected result
TC-41	`CAPTCHA_DISABLED=true`, `SIGNUP_LIMIT_PER_MINUTE=3`	Sign up 4 times within one minute from the same network (e.g. office Wi-Fi)	Calls 1–3 return **201**; call 4 returns **429** `Too many requests from this network...`. The response has a `retry-after-signupPerMinute` header (seconds to wait). Allowed calls carry `x-ratelimit-remaining-signupPerMinute`.
TC-42	Same as TC-41	While the office network is still blocked, sign up from a **different network** (e.g. a phone hotspot)	**201**. *This proves nginx passes the real client IP.* If it's also 429, every user is sharing nginx's IP: fix Part C4.
TC-43	Same as TC-41	From the blocked network, try to fake a new IP with curl (example below)	Still **429**: a client's own `X-Forwarded-For` value is ignored.
TC-44	`LOGIN_LIMIT_PER_15_MINUTES=3`	Log in 4 times from the same network	Calls 1–3 return 200 (or 401 for a wrong password); call 4 returns **429**. A different network can still log in.
TC-45	Any of the low limits above	Call normal APIs (e.g. `GET /student/`, Google login, admin pages) many times	**Never 429**: only the two Student ID routes are rate limited.
TC-46	Original values restored	Sign up and log in normally	Works as in G3.1–G3.5.

Curl for TC-43:

```
curl -i -X POST "https://dev.api.zuvy.org/auth/student/signup" \
  -H "Content-Type: application/json" \
  -H "X-Forwarded-For: 1.2.3.4" \
  -d '{"name":"Spoof Test","password":"test@123","state":"Andaman and Nicobar
Islands","city":"Andamans","udiseCode":"35010300101"}'
```

G3.7 Enrolment by org admins and ops

Use the org admin and ops tokens: log in to the dev web app as each user and copy the token, as in Part E4. "Own course" means a private course of the organisation the user is **currently** logged into.

ID	Caller	Steps	Expected result
TC-47	Org **admin**	Enrol a Student ID student into an **own** course with a `batchId`	**200**; `data.name` shows the student; as the student, `GET /student/` lists the course.
TC-48	Org **ops**	Same as TC-47, with another Student ID student	**200**.

ID	Caller	Steps	Expected result
TC-49	Org **admin** or **ops**	Enrol into a course of **another** organisation	**403** `Only super admins, or admins and ops of this course's organisation, can enrol students by Student ID`. Nothing is created.
TC-50	**Instructor**	Enrol into a course they teach	**403**, same message as TC-49.
TC-51	Org **admin** in 2 organisations	Switch org (`POST /org/switch-org`) to org B, enrol into an org A course; then switch back to org A and retry with the new token	First **403**, then **200**: only the **current** organisation counts.
TC-52	Org **admin**	After TC-47 and TC-49, open the organisation's activity log (`GET /trackinglog`)	An **enroll_student** entry with the admin as actor, the Student ID and the course name. TC-49 appears as a **failed** entry.

G3.8 Session ID

Use the database query "Sessions of a user" from Part H to check the rows.

ID	Steps	Expected result
TC-65	Log in twice as the TC-01 student (TC-06, then again)	Two **different** `sessionId` values. Two rows in `zuvy_user_sessions` with `login_method = student_id`, the real client IP and the browser.
TC-66	`POST /auth/refresh` with the latest `refresh_token`	**200**; `sessionId` is the **same** as the login's. That row's `last_refreshed_at` is now filled; no new row.
TC-67	Decode the new `access_token` in the browser console (don't paste tokens into websites): `JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))`	The payload has `"sid"` equal to the `sessionId`.
TC-68	`POST /auth/logout` with that access token	**201**. That row's `ended_at` is now filled. The other session from TC-65 still has `ended_at` empty.
TC-69	As an admin in 2 organisations, log in with Google, then `POST /org/switch-org`	The login response has a `sessionId` and a row with `login_method = google`. The switch response has the **same** `sessionId`; no new row.

G4. Regression tests for users with email

These confirm that nothing changed for existing users. **Run them before every release to production.**

ID	Steps	Expected result
RG-01	Log in to the web app with Google as a **student**	Works. Courses, progress and leaderboard look normal.
RG-02	Log in with Google as an **instructor**	Works. Instructor dashboard and classes load.

ID	Steps	Expected result
RG-03	Log in with Google as an **admin**	Works. Courses, batches and students load.
RG-04	Log in with Google as a **super admin**	Works.
RG-05	Check the login response (Network tab) for an org **POC** user	`isPoc` is `true` for the POC and `false` for everyone else, the same as before.
RG-06	As an admin in 2 organisations, switch org (`POST /org/switch-org`)	Works and returns new tokens.
RG-07	Stay idle until the token refreshes, or call `POST /auth/refresh`	Works.
RG-08	Log out, then try the old token	401, as before.
RG-09	Create a Google Meet class and a Zoom class for a normal batch	Both are created, and invites are sent to all students.
RG-10	After a Zoom class, check attendance	Recorded for students who attended, as before.
RG-11	Merge two sessions (`POST /classes/merge`)	Works; child-batch students are added to the parent session.
RG-12	Add students by email to a course (`POST /bootcamp/students/{id}`)	Works as before.
RG-13	Google login (any role): look at the login response	Same fields as before, **plus** `sessionId`. Nothing else changed.
RG-14	A user who was already logged in **before** the deploy keeps using the app	Still works; their next refresh returns a `sessionId` (a new session starts, `login_method = token_refresh`).

Part H: Checking the database

Run `SET search_path TO main;` (or `stage_template`) first.

Latest Student ID accounts:

```
SELECT c.student_id, u.id AS user_id, u.name, u.email, u.google_user_id, u.mode,
       c.state, c.city, c.school_name, c.udise_code,
       c.failed_attempts, c.locked_until, c.last_reset_by, c.last_reset_at, c.created_at
FROM zuvy_student_credentials c
JOIN users u ON u.id = c.user_id
ORDER BY c.created_at DESC
LIMIT 10;
```

Expected: `email` and `google_user_id` are NULL, `mode` is `student`, and new accounts have `state`, `city`, `school_name` and `udise_code` filled in.

Students per school:

```
SELECT udise_code, school_name, city, state, COUNT(*) AS students
FROM zuvy_student_credentials
WHERE udise_code IS NOT NULL
GROUP BY udise_code, school_name, city, state
ORDER BY students DESC;
```

The school list:

```
-- After the import: 13,72,195 schools in 35 states
SELECT COUNT(*) AS schools, COUNT(DISTINCT state) AS states FROM zuvy_schools;

-- How many schools per state and district
SELECT state, district, COUNT(*) AS schools,
       COUNT(*) FILTER (WHERE NOT is_active) AS inactive
FROM zuvy_schools
GROUP BY state, district
ORDER BY state, district;

-- Find a school by name or code
SELECT udise_code, school_name, village, block, state, district, is_active
FROM zuvy_schools
WHERE school_name ILIKE '%ramakrishnapur%' OR udise_code = '35010300101';

-- Hide a school from the dropdowns (and refuse it at sign-up)
UPDATE zuvy_schools SET is_active = false, updated_at = now()
WHERE udise_code = '99000000003';

-- Hide the QA test schools from G3.0 after testing
UPDATE zuvy_schools SET is_active = false, updated_at = now()
WHERE udise_code LIKE '9900000000_';
```

A later CSV upload or import that contains a hidden school makes it active again. The API's state and city lists refresh within 10 minutes.

Sessions of a user (any user, Google or Student ID):

```
SELECT s.id AS session_id, s.login_method, s.org_id, s.ip, s.created_at,
       s.last_refreshed_at, s.ended_at
FROM zuvy_user_sessions s
WHERE s.user_id = 48213          -- or: (SELECT user_id FROM zuvy_student_credentials WHERE student_id
= 'riyasharmact1')
ORDER BY s.created_at DESC
LIMIT 20;
```

Logins per day, by method:

```
SELECT created_at::date AS day, login_method, COUNT(*) AS logins,
       COUNT(DISTINCT user_id) AS users
FROM zuvy_user_sessions
GROUP BY 1, 2
ORDER BY 1 DESC, 2;
```

Passwords are never stored in plain text:

```
SELECT student_id, LEFT(password_hash, 20) AS hash_start
FROM zuvy_student_credentials
ORDER BY created_at DESC
LIMIT 5;
```

Expected: every value starts with `scrypt$16384$8$1$`.

Enrolments of a Student ID account:

```
SELECT e.bootcamp_id, e.batch_id, e.status, e.enrolled_date
FROM zuvy_batch_enrollments e
JOIN zuvy_student_credentials c ON c.user_id = e.user_id
WHERE c.student_id = 'riyasharmact1';
```

Unlock an account by hand (the reset API does the same):

```
UPDATE zuvy_student_credentials
SET failed_attempts = 0, locked_until = NULL
WHERE student_id = 'riyasharmact1';
```

Delete test accounts (deleting the user also deletes the credential row):

```
DELETE FROM users
WHERE id IN (
  SELECT user_id FROM zuvy_student_credentials
  WHERE student_id IN ('riyasharmact1')
);
```

If the delete fails because of rows in other tables (enrolments, submissions and so on), remove those test rows first, or just leave the test accounts on dev.

Part I: Known limitations

Limitation	Workaround or plan
Live-class attendance is not recorded automatically for Student ID students. Zoom and Google Meet identify people by email, and these students join as guests.	An admin marks them with `POST /bootcamp/attendance/mark` (TC-36).
Students without an email get **no calendar or Zoom invites**.	They join from the class link in the Zuvy app.

Limitation	Workaround or plan
Old sessions stay valid after a password reset, until the tokens expire (up to 7 days through refresh). Student tokens are not stored on the server, so they can't be cancelled.	Acceptable for forgotten passwords.
Student IDs are **predictable** (name + number), so a classmate could guess another student's ID.	The password is what protects the account: 5 wrong passwords lock that account for 15 minutes, and the per-IP limit stops mass guessing. Ask students to keep their password private.
Spelling variants become separate series: "Ravi", "Ravee" and "Raavi" give `ravict1`, `raveect1` and `raavict1`.	Harmless, because the student is shown their exact ID, but a child may later misremember the spelling. The Student ID card (Part D8) and the super admin search by name (Part F4) cover this.
A student who forgets their Student ID may sign up again and get a second account, splitting their progress.	Frontend: Student ID card + remembered ID on the device (D8). Later: match on date of birth and school.
Rate-limit counters are kept **in the container's memory**. They reset when the container restarts, and if the API is scaled to several containers, each one counts separately.	Fine for one container. For several, switch the throttler to Redis storage.
A very large school (more than about 300 sign-ups an hour from one public IP) could reach the sign-up ceiling.	Raise `SIGNUP_LIMIT_PER_HOUR` / `SIGNUP_LIMIT_PER_MINUTE` in `.env`. No code change is needed.
Sign-up depends on Cloudflare. If Cloudflare is unreachable, sign-up returns 503. Login is not affected.	Rare. Students can retry a few minutes later.
A determined person can still solve captchas by hand and create accounts slowly.	Watch for spikes. Class join codes with seat limits would stop this completely (future work).
These accounts **must stay students**. Giving them an admin or instructor role breaks their login, because staff sessions are stored with an email column that can't be empty.	Don't assign roles to Student ID accounts.
Only **super admins** can reset passwords and search Student ID accounts. (Enrolment is also open to org admins and ops.) Searching is not org-limited, because these students belong to no organisation until enrolled.	Org admins ask a super admin for resets and lookups; for enrolment they ask the student or teacher for the Student ID (from the ID card).
The existing **"add students" API and CSV upload** still work only by email.	Use "Add by Student ID" (Part D12 / F5) for these students.
There is no parental consent step, which the DPDP Act requires for under-18 learners.	Product or legal to decide (for example a consent checkbox on sign-up).
The school list is a **snapshot** of the UDISE data on the India Data Portal (13,72,195 operational schools, updated there about September 2024). Schools opened later are missing; closed schools stay listed until hidden.	A super admin adds a missing school (Part C9) when a student asks. Re-run the import when a newer file is published (Part C8); it updates, never duplicates.
The file has no schools for **Dadra and Nagar Haveli and Daman and Diu**, so that union territory isn't in the state dropdown.	Add its schools with a CSV upload (Part C9) if Zuvy needs it.
There is no public API with school names and UDISE codes; the two suggested GitHub projects only hold summary statistics (Part A6).	The list is kept in our own table, loaded from the official file.

Limitation	Workaround or plan
School names are in capital letters, as in UDISE, and sometimes abbreviated (`GPS` , `ZPHS` , `UPS`).	The search also matches the village, block, pincode and UDISE code, and the second line in the list (Part D1) helps the student recognise their school.
Colleges are not in the list (they have no UDISE code).	Can be added later as a separate list.
Student ID accounts created **before** migration 0045 have no state, city or school.	A super admin can see them with `GET /auth/student/admin/students` (empty school fields). Filling them in later would need a small "update my school" API.
A session that ends without a logout (browser closed, token expired) keeps `ended_at` empty.	Use `last_refreshed_at` (or `created_at`) as the last time the session was seen.
Saving the session row is best effort. If the database write fails, the login still works, but that session is missing from `zuvy_user_sessions` (the error is in the logs).	Rare. Look for `Could not record session` in the logs.

Part J: Troubleshooting

You see	Likely cause	Fix
Sign-up returns **500** `Could not create the account`	The migration hasn't been run on this database	Run Part C1
Student ID sign-up **and login** return **500** right after a deploy	Migration `0045` wasn't run (the code reads the new school columns)	Run `0045` (Part C1). Google login is not affected.
The State dropdown is empty / `GET /schools/states` returns `[]`	The school list hasn't been loaded on this environment	Run the import (Part C8). If it was loaded in the last 10 minutes, wait or restart the container.
Import says `The table zuvy_schools is missing or old`	Migration 0045 hasn't run, or an earlier version of it ran	Run the current `0045_schools_and_sessions.sql` (Part C1), then the import again
Import check shows many skipped rows or `Missing column(s)`	The file was opened and saved in Excel, or it's a different file (e.g. "State-wise School Education Indicators")	Download "Basic Details of Schools" again and use it exactly as downloaded
Import loaded into the wrong database	`.env` on that machine points somewhere else	Always read the last line of the check run (Part C8 step 2) before adding `--yes`
Sign-up **400** `School not found...`	The UDISE code isn't in the list, or the school was made inactive	Pick the school from the dropdown again; if it's really missing, add it with a CSV upload
Sign-up **400** `...not in the selected state and city...`	The `state` / `city` sent don't match the school's (e.g. the student changed the city after picking the school)	Send the state and city that the school was listed under
`GET /schools/...` returns **401** `Token not found`	Old middleware without the public school routes	Deploy the latest build

You see	Likely cause	Fix
Upload **403** `Only super admins can upload the school list`	The token isn't a super admin's	Use a super admin token (Part E4)
Upload shows many `skipped` rows	Wrong columns, UDISE codes with spaces or letters, or a spreadsheet that turned codes into numbers like `3.6E+10`	Format the UDISE column as **Text** in Excel before saving, fix the rows listed in `errors`, upload again
The same city appears twice in the dropdown	An upload used a misspelt district (e.g. "Hydrabad"); different letter case alone can't cause it	Upload the same UDISE codes again with the right spelling; they're updated in place
`sessionId` missing from a login response	Old build	Deploy the latest build
Logins work but `zuvy_user_sessions` stays empty	Migration `0045` not run, so saving the session fails (logged as `Could not record session`)	Run `0045` (Part C1)
Every student API returns **401** with a Student ID token	The server is running old code without the middleware change	Deploy the latest build
`/auth/student/signup` returns **401** `Token not found`	Old middleware without the public routes	Deploy the latest build
Reset / search APIs return **403**	The token isn't a super admin's	Use a super admin token (Part E4)
Enrol API returns **403** for an org admin or ops user	The course belongs to another organisation than the one they're currently logged into, or the user is an instructor	Switch to the course's organisation (`POST /org/switch-org`) and use the new token, or ask a super admin
429 on login `Too many wrong attempts...`	The account is locked after 5 wrong passwords	Wait 15 minutes, or reset the password
429 `Too many requests from this network...`	The per-IP limit was reached	Wait the number of seconds in the `retry-after-<limit name>` header, or raise the limit in `.env`
Everyone gets **429** at the same time, from different places	nginx isn't forwarding the client IP, so all users share nginx's IP	Fix nginx (Part C4) and check `TRUST_PROXY`
Sign-up **400** `Captcha is required`	`captchaToken` is missing	Send the token, or set `CAPTCHA_DISABLED="true"` on dev
Sign-up **400** `Captcha verification failed`	The token is invalid, expired (older than 5 minutes), already used, or from another site	Reset the box and get a fresh token. With test keys, leave `TURNSTILE_ALLOWED_HOSTNAMES` empty. Check the site key and secret key are from the same pair.
Sign-up **503** `Sign-up is temporarily unavailable`	`TURNSTILE_SECRET_KEY` isn't set in the running container	Add it to `.env` and recreate the container; check its length (Part C3)

You see	Likely cause	Fix
The Cloudflare box doesn't appear on the page	Wrong site key, the page's domain isn't in the widget's hostnames, or CSP blocks it	Check the site key, the widget hostnames (Part C2) and the CSP (Part D9)
400 `property X should not exist`	The body has an extra field	Send only the fields shown in Part F
Endpoints missing in Swagger	Old build, or you're on production (Swagger is off there)	Use the dev Swagger page after deploying
`npm test` fails with "Directory ... libs ... was not found"	The project's Jest config points to a missing folder	Use the local config in Part E1
`git commit` hangs at `prettier --write` (Windows)	The pre-commit hook's Prettier step can hang on some Windows machines with many files	Stop it (Ctrl+C), run `npx prettier --write <files>`, `git add` them, and ask the team lead how to commit